



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных
технологий (МОСИТ)

КУРСОВАЯ РАБОТА

по дисциплине «Разработка клиент-серверных мобильных приложений»

Тема курсовой работы: «Разработка клиент-серверного мобильного приложения Музыка»

Студент группы ИКБО-65-23 Олефиров Георгий Гурамович

(подпись)

Руководитель
курсовой работы Старший преподаватель
Дворникова Е.М.

(подпись)

Работа представлена к защите «___» _____ 2026 г.

Допущен к защите «___» _____ 2026 г.

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

Утверждаю
И.о. заведующего кафедрой МОСИТ
_____ Головин С.А.
«16» февраля 2026 г.

ЗАДАНИЕ
на выполнение курсовой работы
по дисциплине «Разработка клиент-серверных мобильных приложений»

Студент Олефиров Георгий Гурамович

Группа ИКБО-65-23

Тема «Разработка клиент-серверного мобильного приложения Музыка»

Исходные данные: Разрабатываемый прототип мобильного приложения должен предоставлять возможности полностью интерактивной системы с понятным дружественным UI и обеспечивать необходимую функциональность, которая формируется в зависимости от заданной темы и предметной области изучаемых вопросов.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Описание предметной области мобильного приложения;
2. Сравнительный анализ существующих аналогов мобильного приложения;
3. Обоснование выбора инструментальных средств разработки мобильного приложения;
4. Определение функциональных и нефункциональных требований;
5. Проектирование пользовательского интерфейса мобильного приложения;
6. Логическая модель мобильного приложения (диаграмма в нотации IDEF0, диаграмма классов);
7. Проектирование базы данных мобильного приложения;
8. Разработка клиентской части мобильного приложения;
9. Разработка серверной части мобильного приложения;
10. Тестирование разработанного мобильного приложения.

Отчет по курсовой работе в виде пояснительной записки.

Срок представления к защите курсовой работы:

до «30» мая 2026 г.

Задание на курсовую работу выдал

Дворникова Е.М.

Подпись руководителя

(ФИО руководителя)

«16» февраля 2026 г.

Задание на курсовую работу получил

Олефиров Г.Г.

Подпись обучающегося

(ФИО обучающегося)

«16» февраля 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ	7
1.1. Описание предметной области мобильного приложения	7
1.2. Сравнительный анализ существующих аналогов.....	8
1.3. Обоснование выбора инструментальных средств для разработки мобильного приложения	10
ГЛАВА 2. ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ	13
2.1. Определение функциональных и нефункциональных требований (описание функциональных требований, use case диаграмма, описание нефункциональных требований)	13
2.2. Проектирование пользовательского интерфейса	18
2.2.1. Навигационная схема	25
2.2.2. Скриншоты реализованного приложения	27
2.3. Логическая модель приложения	33
2.3.1 Проектирование мобильного приложения	33
2.3.2 Проектирование базы данных (логическая модель БД, описание каждой таблицы)	43
2.3.3 SQL-скрипты для создания таблиц	46
ГЛАВА 3. РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ	49
3.1 Разработка клиентской части мобильного приложения	49
3.1.1 Архитектура клиентского приложения	49
3.1.2 Настройка темы и Material3	49
3.1.3 Слой локального хранения (Room).....	51
3.1.4 Слой API (Retrofit).....	52
3.1.5 Управление токеном и аутентификацией	53
3.1.6 CatalogViewModel (управление каталогом)	54
3.1.7 CatalogScreen (экран каталога)	57
3.1.8 PlaylistsViewModel (управление плейлистами)	59
3.1.9 PlaybackService (сервис воспроизведения)	61
3.1.10 Навигация.....	63
3.2 Разработка серверной части мобильного приложения.....	65
3.2.1 Аутентификация и авторизация	65

3.2.2 Основная конфигурация сервера	68
3.2.3 API-эндпоинты	71
3.3. Тестирование разработанного приложения	74
ЗАКЛЮЧЕНИЕ	85
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	88
ПРИЛОЖЕНИЯ	91

ВВЕДЕНИЕ

Современные пользователи мобильных устройств всё чаще сталкиваются с проблемой организации своей музыкальной коллекции и управления плейлистами. Большое количество музыкальных сервисов, потоковых платформ и локальных файлов затрудняет отслеживание любимых треков, создание упорядоченных плейлистов и доступ к музыке в офлайн-режиме. Файловые менеджеры и стандартные медиаплееры не справляются с этой задачей, а существующие решения часто требуют платной подписки или не предоставляют открытого API для интеграции. Приложение MuseFlow призвано решить эти проблемы, предоставляя удобный и современный инструмент для прослушивания музыки и управления плейлистами с возможностью синхронизации данных между устройством и сервером.

Актуальность. С ростом популярности стриминговых сервисов и увеличением объёмов музыкальных библиотек у пользователей возникает потребность в структурированном подходе к организации своей медиатеки. Разработка открытого клиент-серверного музыкального приложения позволит пользователям систематизировать свою коллекцию, быстро находить треки по названию или исполнителю, создавать и редактировать плейлисты, а также слушать музыку в фоновом режиме. При этом отсутствие обязательной подписки и возможность работы офлайн делают приложение доступным для широкого круга слушателей.

В отличие от большинства коммерческих аналогов, MuseFlow предоставляет открытый REST API, что позволяет сторонним разработчикам создавать собственные клиенты. Приложение поддерживает кэширование треков и плейлистов с помощью Room Database, что обеспечивает полноценную работу в офлайн-режиме без необходимости платной подписки.

Цель работы – разработать и протестировать клиент-серверное мобильное приложение для прослушивания музыки и управления плейлистами.

Задачи:

1. Описание предметной области мобильного приложения;
2. Сравнительный анализ существующих аналогов мобильного приложения;
3. Обоснование выбора инструментальных средств разработки мобильного приложения;
4. Определение функциональных и нефункциональных требований;
5. Проектирование пользовательского интерфейса мобильного приложения;
6. Логическая модель мобильного приложения (диаграмма в нотации IDEF0, диаграмма классов);
7. Проектирование базы данных мобильного приложения;
8. Разработка клиентской части мобильного приложения;
9. Разработка серверной части мобильного приложения;
10. Тестирование разработанного мобильного приложения.

Объект исследования – процессы учёта, организации и воспроизведения музыкальных треков, интересующих пользователя.

Предмет исследования – клиент-серверное мобильное приложение для прослушивания музыки и управления плейлистами.

ГЛАВА 1. АНАЛИЗ СОСТОЯНИЯ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.1. Описание предметной области мобильного приложения

Мобильное приложение MuseFlow (клиент-серверная система для прослушивания музыки) предназначено для организации личной музыкальной коллекции пользователя, управления плейлистами и воспроизведения аудиотреков. Цель приложения – предоставить удобный, современный инструмент для доступа к музыке с возможностью синхронизации данных между устройством и сервером, а также для работы в условиях нестабильного интернет-соединения за счёт кэширования.

Предметная область охватывает процессы учёта, поиска, воспроизведения и каталогизации музыкальных произведений. Пользователь взаимодействует с каталогом треков, может добавлять их в собственные плейлисты, редактировать состав плейлистов, искать музыку по названию или исполнителю, а также слушать треки с базовыми функциями плеера (play/pause, следующий/предыдущий трек, перемотка). Все действия, связанные с изменением данных (создание плейлиста, добавление или удаление трека), отправляются на сервер *через* API, после чего клиент обновляет локальный кэш для поддержания актуальности данных в офлайн-режиме., что обеспечивает единое состояние аккаунта на всех устройствах. При отсутствии сети приложение продолжает работать за счёт локального кэша (Room Database), а после восстановления соединения синхронизация происходит автоматически.

Ранее пользователи часто хранили музыкальные коллекции в виде отдельных файлов на устройстве или использовали громоздкие медиаплееры без облачной синхронизации. Это приводило к потере данных при смене телефона, невозможности доступа к плейлистам с других устройств и отсутствию централизованного поиска. MuseFlow решает эти проблемы: сервер хранит всю информацию о треках и плейлистах, а клиентское приложение предоставляет интуитивно понятный интерфейс для управления ими. Благодаря кэшированию

часто используемые треки и списки доступны офлайн, а механизм JWT-аутентификации гарантирует безопасность данных.

Дополнительной ценностью приложения является поиск с сохранением истории (до 10 последних запросов), поддержка тёмной темы интерфейса и встроенный аудиоплеер на базе EchoPlayer, позволяющий слушать музыку в фоновом режиме. Таким образом, предметная область включает три основных аспекта: каталогизация и поиск музыки, управление плейлистами, воспроизведение с синхронизацией состояния. MuseFlow превращает хаотичное хранение аудиофайлов в упорядоченную медиатеку, доступную в любой момент и с любого устройства.

1.2. Сравнительный анализ существующих аналогов

Для выявления сильных и слабых сторон существующих решений, а также для обоснования архитектурных решений MuseFlow, был проведён сравнительный анализ трёх популярных музыкальных сервисов: Яндекс Музыка, VK Музыка и Apple Music. Критерии сравнения отражают ключевые функциональные требования: платформы, наличие поиска с историей, управление плейлистами (CRUD), поддержка офлайн-режима, открытость API, технологический стек клиента и сервера, а также аутентификация.

Яндекс Музыка – один из лидеров российского рынка стриминга. Предлагает обширный каталог, персональные рекомендации, офлайн-режим по подписке, возможность создания и редактирования плейлистов. Однако API для сторонних разработчиков официально не предоставляется, что исключает интеграцию с другими приложениями. Технологический стек закрыт, но известно, что мобильные клиенты написаны на Kotlin с использованием Jetpack Compose (в новых версиях) и MVVM.

VK Музыка – встроенный музыкальный сервис экосистемы VK. Предлагает схожий функционал: плейлисты, рекомендации, офлайн-режим по подписке. Также не имеет открытого API. Аутентификация привязана к аккаунту

VK, что может быть неудобно для пользователей без аккаунта в соцсети. Мобильное приложение VK включает музыкальный раздел, но как отдельное приложение VK Музыка существует и развивается отдельно (написано на Kotlin, с использованием собственных фреймворков).

Apple Music – глобальный сервис от Apple. Предлагает интеграцию с экосистемой Apple, качественный каталог, плейлисты, офлайн-режим. API для сторонних разработчиков существует (Apple Music API), но требует регистрации разработчика и соблюдения жёстких правил. Клиентские приложения для Android написаны на Kotlin (с элементами Java), но код закрыт. Аутентификация через Apple ID.

MuseFlow в отличие от перечисленных аналогов является открытым проектом с полным REST API (документация Swagger в разработке), что позволяет сторонним разработчикам создавать собственные клиенты. Он ориентирован на локальное хранение кэша (Room) для работы офлайн, поддерживает историю поиска, тёмную тему и полностью управляется пользователем без подписки. В таблице 1 приведено сравнение по ключевым критериям.

Таблица 1 – Сравнительный анализ существующих аналогов

Критерий	Яндекс Музыка	VK Музыка	Apple Music	MuseFlow
Платформы	Android, iOS, Web	Android, iOS, Web	Android, iOS, macOS, Web	Android
Поиск с историей	Нет (только рекомендации)	Нет	Нет	Да (до 10 запросов)
Управление плейлистами (создание, редактирование, удаление)	Да	Да	Да	Да
Офлайн-режим	Да (по подписке)	Да (по подписке)	Да (по подписке)	Да (бесплатно, кэш треков и плейлистов)
Открытое REST API	Нет	Нет	Ограниченное (требуется регистрация разработчика)	Да (полный CRUD, JWT)
Технологический стек клиента	Kotlin, Compose (закрыто)	Kotlin, собственные фреймворки	Kotlin, Java (закрыто)	Kotlin, Jetpack Compose, Hilt, Room, Retrofit, ExoPlayer

Технологический стек сервера	Закрыт	Закрыт	Закрыт (Apple private)	Ktor, Exposed, PostgreSQL, JWT, BCrypt
Аутентификация	По номеру телефона / соцсети	Через VK ID	Apple ID	Логин + пароль (JWT)

Все рассмотренные коммерческие сервисы предоставляют широкий функционал, но являются проприетарными, не дают возможности интеграции и требуют платной подписки для офлайн-доступа. Таким образом, MuseFlow не стремится конкурировать с коммерческими гигантами по объёму каталога, а предлагает альтернативу для пользователей, ценящих открытость, отсутствие подписки и возможность самостоятельного хостинга.

1.3. Обоснование выбора инструментальных средств для разработки мобильного приложения

Для реализации клиентской части MuseFlow были выбраны следующие технологии:

Kotlin – современный статически типизированный язык, официально рекомендованный Google для разработки под Android. Обеспечивает лаконичность, безопасность типов, поддержку корутин (Coroutines) для асинхронных операций, а также полную совместимость с Java-библиотеками.

Jetpack Compose – декларативный UI-фреймворк от Google. Позволяет создавать интерфейс с меньшим количеством кода по сравнению с XML, обеспечивает реактивную модель обновления и легко интегрируется с StateFlow. Для построения интерфейса используются компоненты Material 3, реализующие современные гайдлайны дизайна.

Dagger Hilt – стандартная библиотека для внедрения зависимостей в Android-приложениях. Упрощает управление зависимостями, уменьшает связность кода и облегчает тестирование. В проекте используются модули для предоставления DAO, репозитория, use case'ов и сетевого клиента. Для навигации используется Hilt Navigation Compose, интегрирующий Hilt с Compose Navigation.

Room – библиотека для локального хранения данных поверх SQLite. Используется для кэширования треков, что обеспечивает возможность поиска по названию и исполнителю в офлайн-режиме. Обеспечивает автоматический маппинг между объектами Kotlin и таблицами БД, поддерживает корутины и Flow для реактивных запросов.

Retrofit + OkHttp + Gson – набор для работы с сетью. Retrofit генерирует реализации HTTP-запросов по аннотированным интерфейсам, OkHttp управляет соединениями и перехватчиками (в частности, для добавления JWT-токена), а Gson преобразует JSON в объекты Kotlin.

ExoPlayer – мощный медиаплеер от Google с встроенной поддержкой потокового воспроизведения, фоновой работы и управления очередью треков. Используется для реализации экрана плеера.

MediaSessionService (AndroidX Media3) – сервис для управления воспроизведением мультимедиа в фоновом режиме. В MuseFlow реализован собственный PlaybackService, наследующий MediaSessionService, который обеспечивает:

- воспроизведение музыки при свёрнутом приложении;

- управление плеером через уведомление (Notification) с кнопками play/pause, next, previous;

- связь между UI и сервисом через Binder (LocalBinder) для передачи команд и получения состояния (текущий трек, позиция воспроизведения, длительность);
- интеграцию с ExoPlayer для формирования очереди треков из плейлиста.

Coil – библиотека для загрузки изображений (обложек треков). Оптимизирована под Kotlin Coroutines и Compose, поддерживает кэширование на диске и в памяти.

Coroutines + StateFlow – для асинхронности и управления состоянием UI. В отличие от LiveData, StateFlow полностью совместим с Compose и обеспечивает более предсказуемое поведение при многопоточности.

Для серверной части выбраны следующие технологии:

Ktor – асинхронный веб-фреймворк на Kotlin, разработанный JetBrains. Позволяет создавать легковесные и производительные серверные приложения. Выбор обусловлен единым языком с клиентом (Kotlin), что упрощает обмен данными и навигацию по коду. Ktor поддерживает встроенную сериализацию JSON через `kotlinx.serialization`, CORS, JWT-аутентификацию и статическую раздачу файлов.

Exposed – легковесный ORM-фреймворк на Kotlin. Позволяет писать SQL-запросы в типобезопасном стиле, не требуя полноценного Hibernate. Удобен для небольших проектов и образовательных целей.

PostgreSQL – мощная объектно-реляционная СУБД с открытым исходным кодом. Используется для хранения пользователей, треков, плейлистов и связей между ними. Выбрана за надёжность, поддержку транзакций и расширенных типов данных.

JWT (auth0 java-jwt) – механизм аутентификации. Сервер выдаёт JSON Web Token после успешного входа, который клиент передаёт в заголовке Authorization. Токен содержит идентификатор пользователя и логин, подписан алгоритмом HMAC256. Это позволяет серверу быть stateless.

BCrypt – библиотека для хеширования паролей. Обеспечивает устойчивость к атакам по радужным таблицам за счёт соли и адаптивной сложности.

Для наглядности все выбранные технологии сгруппированы в таблице 2 по их назначению.

Таблица 2 – Сводка выбранных технологий

Компонент	Технология	Назначение
Язык клиента	Kotlin	Основной язык разработки Android
UI-фреймворк	Jetpack Compose + Material 3	Декларативное построение интерфейса
Внедрение зависимостей	Dagger Hilt	Управление зависимостями, тестирование
Локальное хранение	Room	Кэширование треков (офлайн-доступ)

Сетевые запросы	Retrofit + OkHttp + Gson	HTTP-взаимодействие с сервером
Медиаплеер	ExoPlayer	Воспроизведение аудио
Фоновое воспроизведение	MediaSessionService (Media3)	Управление плеером в фоне
Загрузка изображений	Coil	Отображение обложек треков
Асинхронность	Coroutines + StateFlow	Потокобезопасное управление состоянием UI
Язык сервера	Kotlin (Ktor)	Единый стек с клиентом
ORM для сервера	Exposed	Типобезопасные SQL-запросы
База данных	PostgreSQL	Хранение пользователей, треков, плейлистов
Аутентификация	JWT (auth0) + BCrypt	Stateless-авторизация и хеширование паролей

Все выбранные инструменты являются открытыми, кроссплатформенными и широко применяются в индустрии. Их комбинация обеспечивает высокую производительность, безопасность и удобство разработки клиент-серверного приложения «Музыка».

ГЛАВА 2. ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

2.1. Определение функциональных и нефункциональных требований (описание функциональных требований, use case диаграмма, описание нефункциональных требований)

Функциональные требования формируют набор конкретных возможностей, которые система должна предоставлять пользователю для выполнения его задач:

Регистрация пользователя. Система должна позволять новому пользователю создать аккаунт, указав логин, email и пароль. Пароль хешируется

на сервере с помощью BCrypt. После регистрации пользователь автоматически не входит в систему – требуется явный вход.

Аутентификация по JWT-токену. После успешного входа (логин и пароль) пользователь получает JWT-токен, который действует 7 дней. Токен сохраняется на устройстве в SharedPreferences и передаётся в заголовке Authorization: Bearer <token> при каждом запросе к защищённым эндпоинтам сервера.

Управление плейлистами. Пользователь может создавать новые плейлисты, указывая название (и опционально обложку), переименовывать существующие, удалять их, а также добавлять или удалять треки из любого своего плейлиста. Система не допускает создания двух плейлистов с одинаковым названием у одного пользователя (сервер возвращает статус 409 Conflict).

Каталог треков. Пользователь может просматривать список всех доступных треков с информацией: уникальный идентификатор, название, исполнитель, длительность (в секундах), URL обложки, URL аудиофайла, жанр. Для авторизованного пользователя доступно добавление любого трека в один из его плейлистов.

Поиск треков. Система предоставляет возможность поиска по названию трека или исполнителю. Поиск работает как в онлайн-режиме (через серверный эндпоинт /tracks/search), так и в офлайн-режиме (по кэшированным в Room данным). При выполнении поискового запроса отображается индикатор загрузки (ProgressBar), который скрывается после получения результатов. Поисковый запрос сохраняется при повороте экрана и восстановлении активности. История поиска сохраняется локально на устройстве (не более 10 последних уникальных запросов) с возможностью полной очистки и удаления отдельных запросов. При нажатии на строку поиска отображается список истории (если она не пуста). Кнопка «Очистить» удаляет текущий текст в поле ввода и скрывает клавиатуру. Если поиск не дал результатов, отображается плейсхолдер с сообщением «Ничего не найдено» и кнопкой «Очистить поиск». При ошибке выполнения поискового запроса (отсутствие интернета или ошибка сервера) отображается плейсхолдер с

сообщением об ошибке и кнопкой «Обновить», которая повторяет последний поисковый запрос.

Воспроизведение музыки. Авторизованный пользователь может прослушивать треки с базовыми функциями управления: play/pause, следующий трек, предыдущий трек, перемотка (seek bar). Плеер реализован на базе ExoPlayer и работает в фоновом режиме через MediaSessionService. При воспроизведении трека из плейлиста формируется очередь из всех треков этого плейлиста, что позволяет последовательно переключаться между ними. При воспроизведении одиночного трека (из каталога или поиска) очередь состоит только из него.

Кэширование и офлайн-режим. При наличии интернет-соединения приложение загружает с сервера актуальный список треков и сохраняет их в локальную базу данных Room (таблица tracks). Плейлисты при отсутствии сети не доступны, так как их данные требуют актуальности с сервера. При отсутствии сети (или ошибке соединения) система автоматически переключается на работу с кэшированными данными, отображая их. Пользователь может принудительно очистить кэш через экран профиля.

Редактирование профиля. Пользователь может просматривать свой профиль (отображается логин и email, полученные с сервера через эндпоинт /user/me), а также выходить из аккаунта. Выход приводит к удалению JWT-токена из SharedPreferences и очистке локальной базы данных (опционально – по выбору пользователя через кнопку «Очистить кэш»).

Тёмная тема. Приложение поддерживает ручное переключение между светлой и тёмной темой интерфейса, а также автоматическое следование системной теме (по умолчанию). Переключатель реализован на экране профиля.

Описанные выше возможности пользователя отображены на USE CASE диаграмме на рисунке 1.

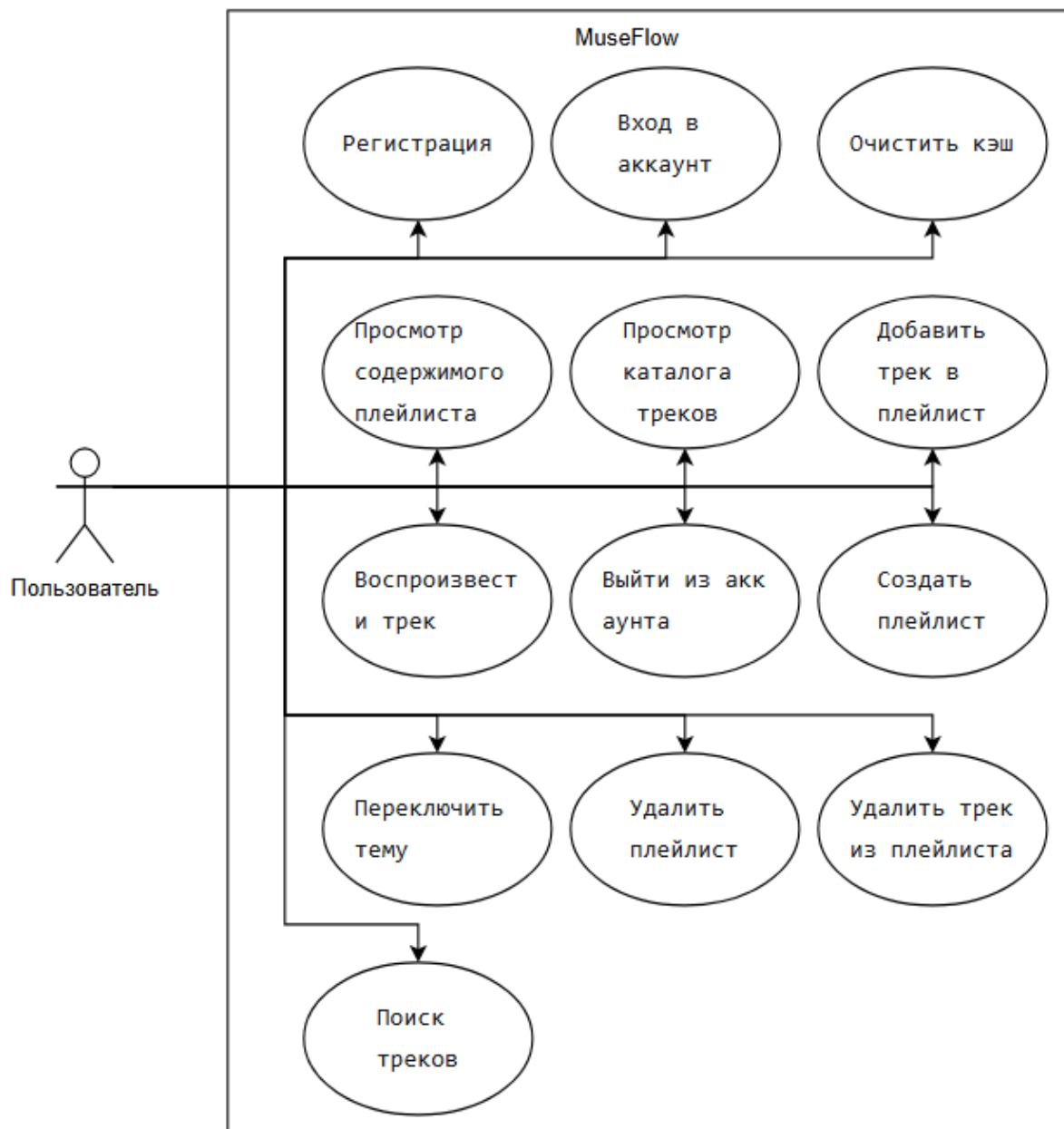


Рисунок 1 Диаграмма USE CASE

Перечень прецедентов (Use Cases) для актора «Пользователь»:

- Зарегистрироваться
- Войти в аккаунт
- Просмотреть каталог треков
- Искать треки (включая: просмотр истории поиска, очистку истории, очистку поля поиска)
- Создать плейлист
- Переименовать плейлист

- Удалить плейлист
- Добавить трек в плейлист
- Удалить трек из плейлиста
- Просмотреть содержимое плейлиста
- Воспроизвести трек (включая: play/pause, следующий трек, предыдущий трек, перемотку)
- Переключить тему интерфейса (светлая/тёмная)
- Очистить кэш
- Выйти из аккаунта

Нефункциональные требования задают критерии качества, безопасности, производительности, надёжности, совместимости, эффективности, удобства использования и масштабируемости системы:

Производительность. Целевое время ответа сервера при нагрузке до 100 одновременных пользователей – не более 200 мсв среднем, 95-й персентиль – до 300 мс. Время загрузки экрана каталога (до 50 треков) – не более 2 секунд при скорости интернета от 10 Мбит/с. Время запуска приложения (холодный старт) – не более 3 секунд на устройстве среднего класса.

Надёжность. Время безотказной работы (MTBF) должно составлять не менее 1000 часов; время восстановления (MTTR) не более 10 минут. Приложение должно корректно работать при потере сети и восстанавливать функциональность после её восстановления без перезапуска

Безопасность. Все соединения с сервером в реальной эксплуатации должны использовать HTTPS (в разработке допустим HTTP для эмулятора 10.0.2.2). Пароли пользователей хранятся на сервере только в виде BCrypt-хэшей (соль 12 раундов). JWT-токен подписывается алгоритмом HMAC256 с секретным ключом, срок действия 7 дней. Сервер при каждом защищённом запросе проверяет подпись токена и извлекает userId для авторизации. Запрещён доступ к чужим плейлистам: при запросе плейлистов сервер сверяет userId из токена с владельцем плейлиста (user_id в таблице playlists). Токен не хранится на сервере, что соответствует stateless-архитектуре.

Совместимость. Клиентское приложение должно работать на устройствах с Android 8.0 (API 26) и выше. Сервер запускается на JVM 21 (Ktor) и может быть развёрнут на любой операционной системе, поддерживающей Java (Windows, Linux, macOS). Для разработки и отладки используется эмулятор Android с доступом к серверу через специальный адрес 10.0.2.2.

Масштабируемость. Архитектура сервера является stateless (JWT не хранится на сервере, все данные о сессии – внутри токена). Это позволяет горизонтально масштабировать серверные экземпляры при общей PostgreSQL базе данных. Клиентское приложение построено на Clean Architecture (слои data, domain, presentation) с использованием Dagger Hilt для внедрения зависимостей, что упрощает добавление новых функций и тестирование.

Поддерживаемость. Код клиента и сервера написан на языке Kotlin с соблюдением стандартов именования и структурирования. Для клиента используется система сборки Gradle (Kotlin DSL), для сервера – встроенная сборка Ktor. Конфигурационные параметры (URL сервера, таймауты соединения, настройки кэша Room, размер истории поиска) вынесены в отдельные файлы: для клиента – в BuildConfig через gradle.properties, для сервера – через переменные окружения или файл конфигурации.

2.2. Проектирование пользовательского интерфейса

Проектирование пользовательского интерфейса мобильного приложения MuseFlow выполнено в графическом редакторе Figma. Были разработаны макеты шести основных экранов в едином стиле Material Design 3 с тёмной темой оформления. Шрифты – стандартные Material 3 (Roboto).

На рисунке 2.1 представлен общий макет всех экранов приложения, демонстрирующий визуальный стиль, компоновку элементов и навигационные переходы.

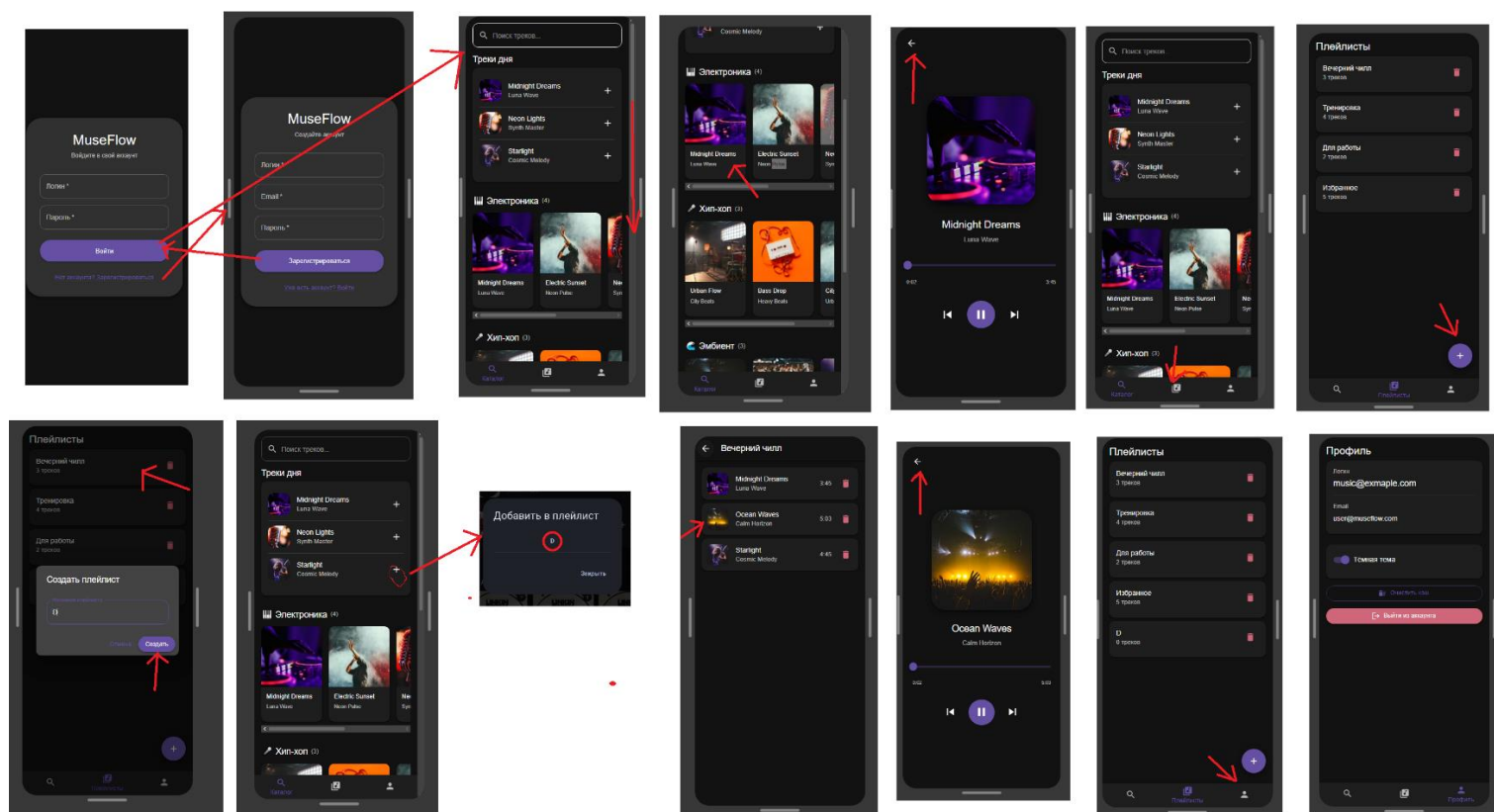


Рисунок 2.1. Макет экранов в Figma

Описание межэкранного взаимодействия

Навигация в приложении MuseFlow построена с использованием библиотеки Jetpack Compose Navigation. Единый NavHost расположен в MainActivity и управляет переходами между всеми экранами. Ниже подробно описывается логика работы каждого экрана и переходы между ними.

Экран авторизации (AuthScreen)

При запуске приложения выполняется проверка наличия сохранённого JWT-токена в SharedPreferences (через TokenManager). Если токен присутствует и не истёк, пользователь автоматически перенаправляется на главный экран (MainScreen), минуя авторизацию. Если токен отсутствует, отображается AuthScreen.

Экран авторизации имеет два режима: «Вход» и «Регистрация», переключаемые нажатием на текстовую ссылку внизу экрана. В режиме «Вход» отображаются поля «Логин» и «Пароль», а также кнопка «Войти». В режиме

«Регистрация» добавляется поле «Email», а кнопка меняет название на «Зарегистрироваться».

При нажатии на кнопку выполняется валидация заполнения полей. Затем вызывается соответствующий метод `AuthViewModel.login()` или `AuthViewModel.register()`, который через `LoginUseCase` или `RegisterUseCase` отправляет запрос на сервер. На время выполнения запроса отображается `CircularProgressIndicator` (лоадер), а кнопка становится неактивной.

При успешной аутентификации сервер возвращает JWT-токен, который сохраняется через `TokenManager.saveToken()`. После этого происходит навигация на `MainScreen` с полной очисткой back-стека (`popUpTo("auth") { inclusive = true }`), чтобы пользователь не мог вернуться на экран авторизации кнопкой «Назад».

При ошибке (неверный логин/пароль, пользователь уже существует, некорректный email и т.д.) в `AuthViewModel` устанавливается состояние `AuthState.Error`, и под кнопкой отображается текст с сообщением об ошибке.

Главный экран (`MainScreen`)

`MainScreen` является контейнером с нижней навигационной панелью (`NavigationBar`). При его создании инициализируются `CatalogViewModel` и `PlaylistsViewModel`, которые загружают начальные данные (треки и плейлисты). Также проверяется наличие токена для отображения информации о пользователе на экране профиля.

Нижняя навигационная панель содержит три пункта:

«Каталог» (`catalog`) – главный экран с треками и поиском;

«Плейлисты» (`playlists`) – экран управления плейлистами;

«Профиль» (`profile`) – экран с настройками и информацией о пользователе.

Каждый пункт связан с соответствующим экраном через `NavHost`. Переключение между экранами происходит без пересоздания `ViewModel` благодаря использованию `rememberNavController()` и правильной структуре `NavHost`.

Экран каталога (`CatalogScreen`)

CatalogScreen является основным экраном приложения и содержит следующие зоны:

Верхняя панель поиска: В верхней части экрана расположено поле ввода (OutlinedTextField) с иконкой лупы. При вводе текста с задержкой (debounce) вызывается CatalogViewModel.search(query), который отправляет запрос на сервер через SearchTracksUseCase. При отсутствии интернет-соединения поиск выполняется по кэшированным в Room данным. Кнопка «Очистить» (крестик) отображается только при наличии текста в поле и при нажатии очищает поле, скрывает клавиатуру и сбрасывает результаты поиска.

История поиска: При получении фокусом поля поиска и пустом значении запроса отображается карточка «Недавние поиски». В ней представлен список последних запросов (не более 10 уникальных), хранящихся в SharedPreferences. Каждый элемент истории содержит текст запроса, при нажатии на который выполняется поиск. Справа от каждого запроса расположена иконка крестика для удаления отдельного элемента из истории. Внизу карточки находится кнопка «Очистить всё», полностью удаляющая историю поиска. При вводе текста или потере фокуса карточка истории скрывается.

Блок «Треки дня»: Под поисковой строкой расположен горизонтальный блок с треками. Треки разбиты на страницы по 3 трека на страницу. Реализована горизонтальная прокрутка с индикатором страниц в виде точек внизу. Каждый трек в блоке отображается с обложкой, названием, исполнителем и иконкой «+» для добавления в плейлист.

Блоки по жанрам: Далее следуют блоки для каждого жанра (рок, поп, хип-хоп и т.д.). Каждый блок начинается с заголовка, содержащего иконку папки, название жанра и количество треков. Далее расположена горизонтальная прокрутка карточек треков (ширина каждой карточки 140 dp). В конце каждого блока находится карточка «Все», при нажатии на которую открывается GenreTracksScreen, отображающий все треки выбранного жанра.

Обработка пустых состояний и ошибок: При пустом результате поиска отображается плейсхолдер с иконкой папки, текстом «Ничего не найдено по

запросу» и кнопкой «Очистить поиск». При ошибке загрузки (отсутствие интернета и пустой кэш, ошибка сервера) отображается плейсхолдер с сообщением об ошибке и кнопкой «Обновить», которая повторяет последний запрос (загрузку каталога или поиск).

Добавление трека в плейлист: При нажатии на иконку «+» рядом с треком открывается диалог `AddToPlaylistDialog`, отображающий список всех плейлистов пользователя. При выборе плейлиста вызывается `PlaylistsViewModel.addToPlaylist(playlistId, trackId)`. Если добавление успешно, показывается `Toast` «Трек добавлен в плейлист». Если трек уже присутствует в плейлисте, сервер возвращает код 409, и пользователь видит сообщение «Трек уже есть в этом плейлисте». При ошибке сети или сервера отображается соответствующее сообщение.

Переход к плееру: При нажатии на карточку трека вызывается `onTrackClick`, который передаёт выбранный трек и весь текущий список треков (весь каталог или результаты поиска) в `MainActivity`. Затем осуществляется навигация на `PlayerScreen` с передачей идентификатора трека через маршрут `"player/{trackId}"`.

Экран плейлистов (`PlaylistsScreen`)

`PlaylistsScreen` отображает все плейлисты текущего пользователя. В правом нижнем углу расположена плавающая кнопка (`FloatingActionButton`) с иконкой «+» для создания нового плейлиста.

Создание плейлиста: При нажатии на `FAB` открывается диалог `CreatePlaylistDialog` с полем ввода названия и кнопками «Создать» и «Отмена». При подтверждении вызывается `PlaylistsViewModel.createPlaylist(name)`. Если плейлист с таким именем уже существует у данного пользователя, сервер возвращает ошибку 409, и отображается `Toast` с сообщением «Плейлист с таким именем уже создан». При успешном создании список плейлистов автоматически обновляется.

Список плейлистов: Каждый элемент списка (`PlaylistItem`) представляет собой карточку, содержащую название плейлиста, количество треков в нём и

иконку корзины для удаления. При нажатии на саму карточку (не на иконку корзины) выполняется переход на `PlaylistDetailScreen` с передачей объекта `Playlist`. При нажатии на иконку корзины вызывается `PlaylistsViewModel.deletePlaylist(playlistId)`, после чего список плейлистов обновляется.

Пустое состояние: Если у пользователя нет плейлистов, отображается плейсхолдер с текстом «Нет плейлистов» и кнопкой «Создать первый плейлист», при нажатии на которую открывается диалог создания плейлиста.

Экран детального просмотра плейлиста (`PlaylistDetailScreen`)

`PlaylistDetailScreen` отображает содержимое выбранного плейлиста. Верхняя панель (`AppBar`) содержит название плейлиста и кнопку «Назад» для возврата к списку плейлистов.

Список треков: Каждый трек в плейлисте отображается в виде карточки с обложкой (50×50 dp), названием, исполнителем, длительностью и иконкой корзины для удаления. При нажатии на карточку трека вызывается `onTrackClick`, который передаёт выбранный трек и весь список треков плейлиста в `MainActivity`. Затем осуществляется навигация на `PlayerScreen` с формированием очереди из всех треков плейлиста.

Удаление трека: При нажатии на иконку корзины вызывается `PlaylistsViewModel.removeTrackFromPlaylist(playlistId, trackId)`. После успешного удаления список треков плейлиста обновляется (через повторную загрузку плейлистов), и пользователь видит `Toast` с подтверждением.

Пустое состояние: Если плейлист не содержит треков, отображается плейсхолдер с иконкой плейлиста и текстом «В плейлисте пока нет треков. Добавьте треки из каталога».

Экран плеера (`PlayerScreen`)

`PlayerScreen` реализует полноценный аудиоплеер на базе `ExoPlayer` и `MediaSessionService`. Экран содержит следующие элементы:

Верхняя панель: Кнопка «Назад» в левом верхнем углу для возврата на предыдущий экран.

Обложка: В центральной части экрана расположена карточка с обложкой текущего трека размером 250×250 dp, скруглёнными углами и поднятой тенью.

Информация о треке: Под обложкой отображаются название трека (крупный шрифт) и имя исполнителя.

Ползунок перемотки (Seek Bar): Под информацией о треке расположен горизонтальный ползунок, отображающий текущую позицию воспроизведения. Пользователь может перематывать трек, перетаскивая ползунок. Под ползунком отображаются текущее время и общая длительность трека в формате MM:SS.

Кнопки управления: В нижней части экрана расположены три кнопки: «Предыдущий трек», «Play/Pause» (крупная круглая кнопка с акцентным цветом), «Следующий трек». При воспроизведении трека из плейлиста кнопки переключения активны и позволяют перемещаться по очереди треков. При воспроизведении одиночного трека кнопки переключения также активны, но не меняют трек (или зацикливают воспроизведение).

Фоновое воспроизведение: Плеер продолжает работу при сворачивании приложения благодаря `PlaybackService`, наследующему `MediaSessionService`. В уведомлении отображаются кнопки управления (play/pause, следующий, предыдущий), обложка и информация о треке.

Получение состояния: `PlayerScreen` подключается к `PlaybackService` через `ServiceConnection` и `LocalBinder`, получая актуальную информацию о текущем треке, позиции воспроизведения и состоянии (играет/на паузе). Обновление состояния происходит каждые 500 мс.

Экран профиля (`ProfileScreen`)

`ProfileScreen` содержит настройки пользователя и информацию об аккаунте. При создании экрана выполняется запрос к эндпоинту `/user/me` для получения логина и email пользователя (через `UserRouting` на сервере).

Информация о пользователе: В карточке отображается логин пользователя, полученный из `TokenManager` (сохраняется при входе). Email не отображается, так как для этого потребовался бы отдельный запрос к серверу. Переключатель темы: Расположен отдельной строкой с иконкой (солнце/луна) и `Switch`. При

переключении темы значение сохраняется в `SharedPreferences` и применяется через `MuseFlowTheme(darkTheme = isDarkTheme)`, который оборачивает всё приложение в `MainActivity`. Изменение темы происходит без перезапуска приложения благодаря реактивной природе `Compose`.

Кнопка «Очистить кэш»: При нажатии вызывается `tracksRepository.clearCache()`, удаляющая все треки из локальной базы данных `Room`. После очистки показывается `Toast` «Кэш очищен». При следующей загрузке каталога данные будут загружены с сервера заново.

Кнопка «Выйти из аккаунта»: При нажатии вызывается `tokenManager.clearToken()`, удаляющая JWT-токен из `SharedPreferences`. После этого выполняется навигация на `AuthScreen` с полной очисткой `back`-стека, чтобы пользователь не мог вернуться в защищённую часть приложения.

Экран треков по жанру (`GenreTracksScreen`)

Дополнительный экран, открывающийся при нажатии на карточку «Все» в блоке жанра или на заголовок жанра на экране каталога. Экран содержит верхнюю панель с названием жанра и количеством треков, кнопку «Назад» для возврата в каталог и вертикальный список всех треков выбранного жанра. Каждый трек отображается аналогично треку в плейлисте (обложка, название, исполнитель, длительность). При нажатии на трек выполняется переход на `PlayerScreen` с очередью из всех треков данного жанра.

2.2.1. Навигационная схема

На рисунке 2.2 представлена схема навигации между экранами приложения `MuseFlow`, демонстрирующая все возможные переходы.

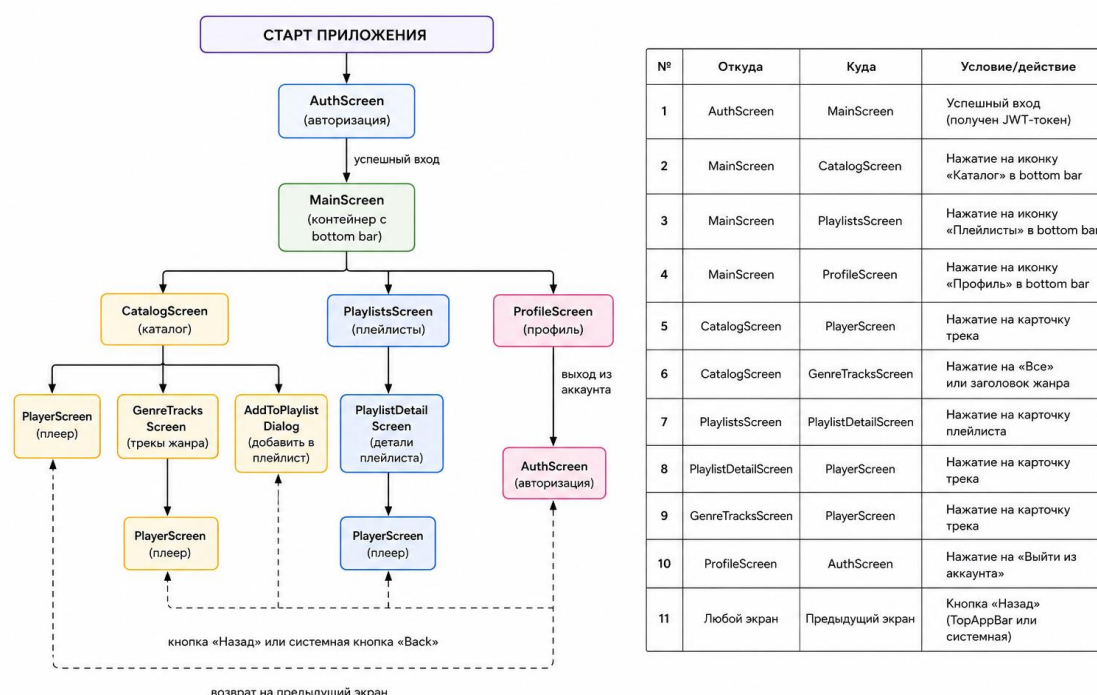


Рисунок 2.2 – Схема навигации между экранами MuseFlow

На схеме отражены следующие переходы:

AuthScreen следует в MainScreen (при успешной авторизации)

MainScreen следует в CatalogScreen / PlaylistsScreen / ProfileScreen (через нижнюю навигацию)

CatalogScreen следует в PlayerScreen (при нажатии на трек)

CatalogScreen следует в GenreTracksScreen (при нажатии на «Все» или заголовок жанра)

PlaylistsScreen следует в PlaylistDetailScreen (при нажатии на плейлист)

PlaylistDetailScreen следует в PlayerScreen (при нажатии на трек)

GenreTracksScreen следует в PlayerScreen (при нажатии на трек)

ProfileScreen следует в AuthScreen (при выходе из аккаунта)

Все экраны следует в предыдущий экран (через кнопку «Назад» или системную кнопку)

2.2.2. Скриншоты реализованного приложения

На рисунках 2.3–2.12 представлены скриншоты всех экранов работающего приложения MuseFlow, демонстрирующие соответствие реализованного интерфейса макетам Figma.



Рисунок 2.3. – Экран авторизации (режимы «Вход» и «Регистрация»)



Рисунок 2.4. – Экран каталога (треки дня, блоки по жанрам)

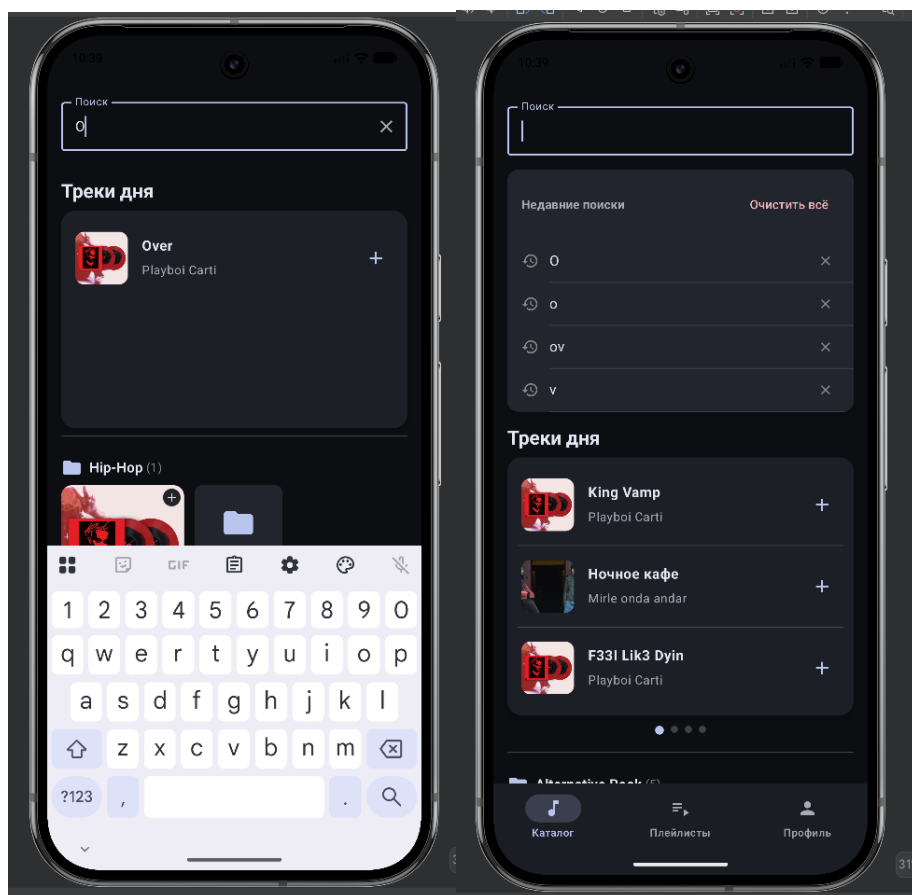


Рисунок 2.5. – Экран каталога (поиск, история поиска)

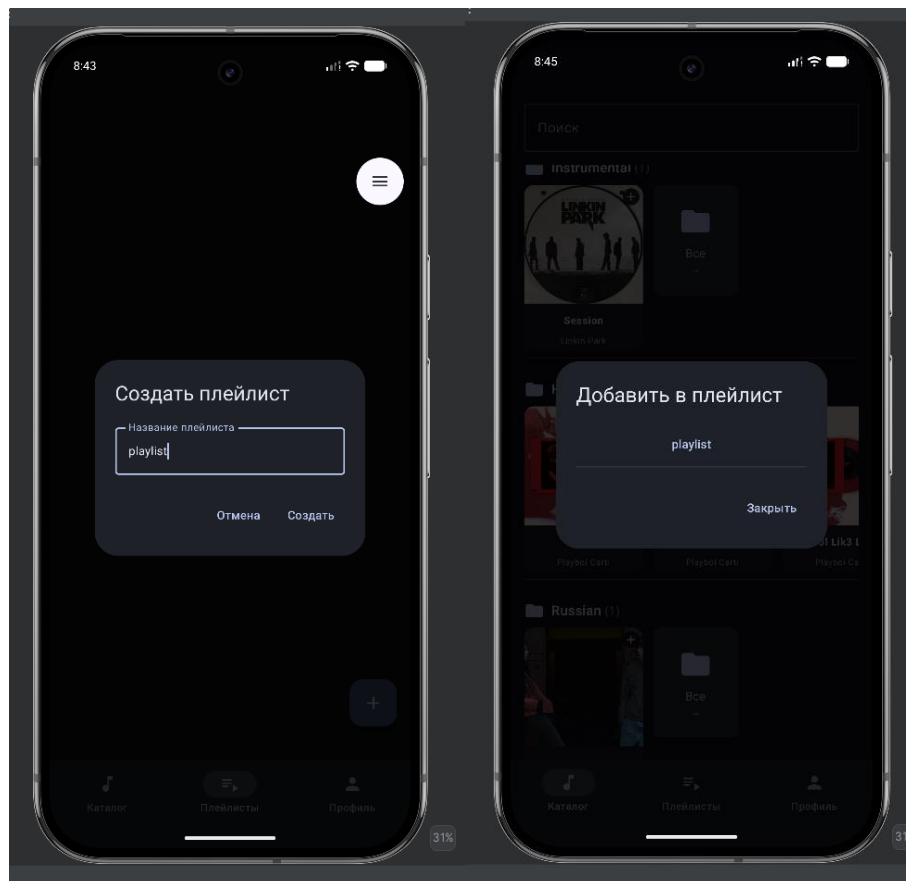


Рисунок 2.6. – Экран плейлистов (список плейлистов и диалог создания)

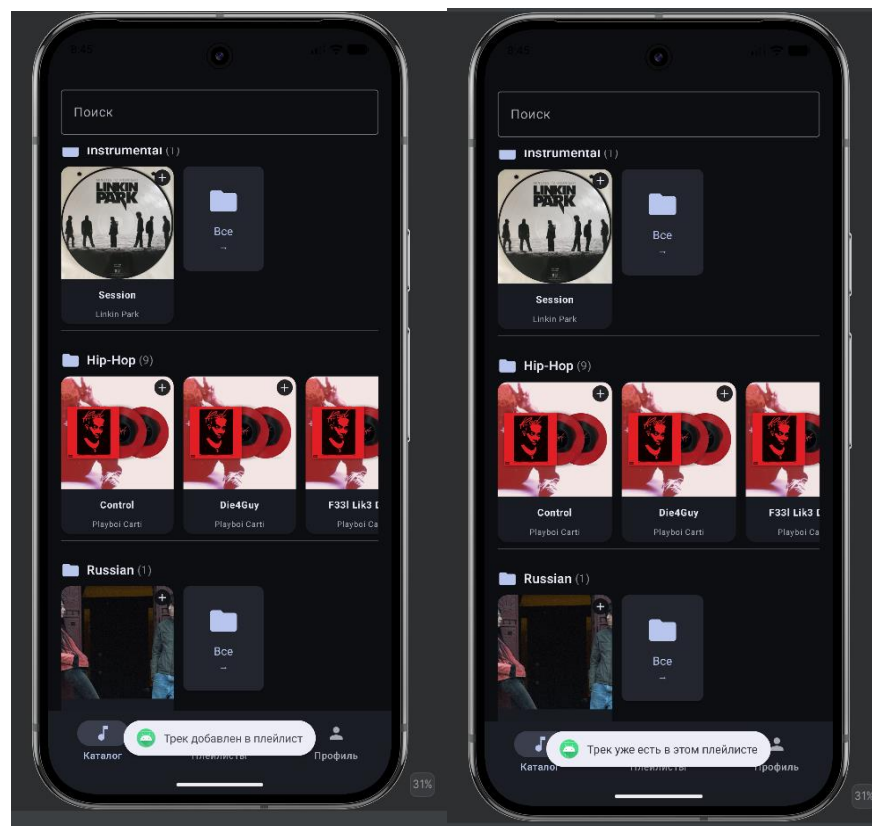


Рисунок 2.7. – Экран плейлистов (уведомление при добавлении в плейлист)

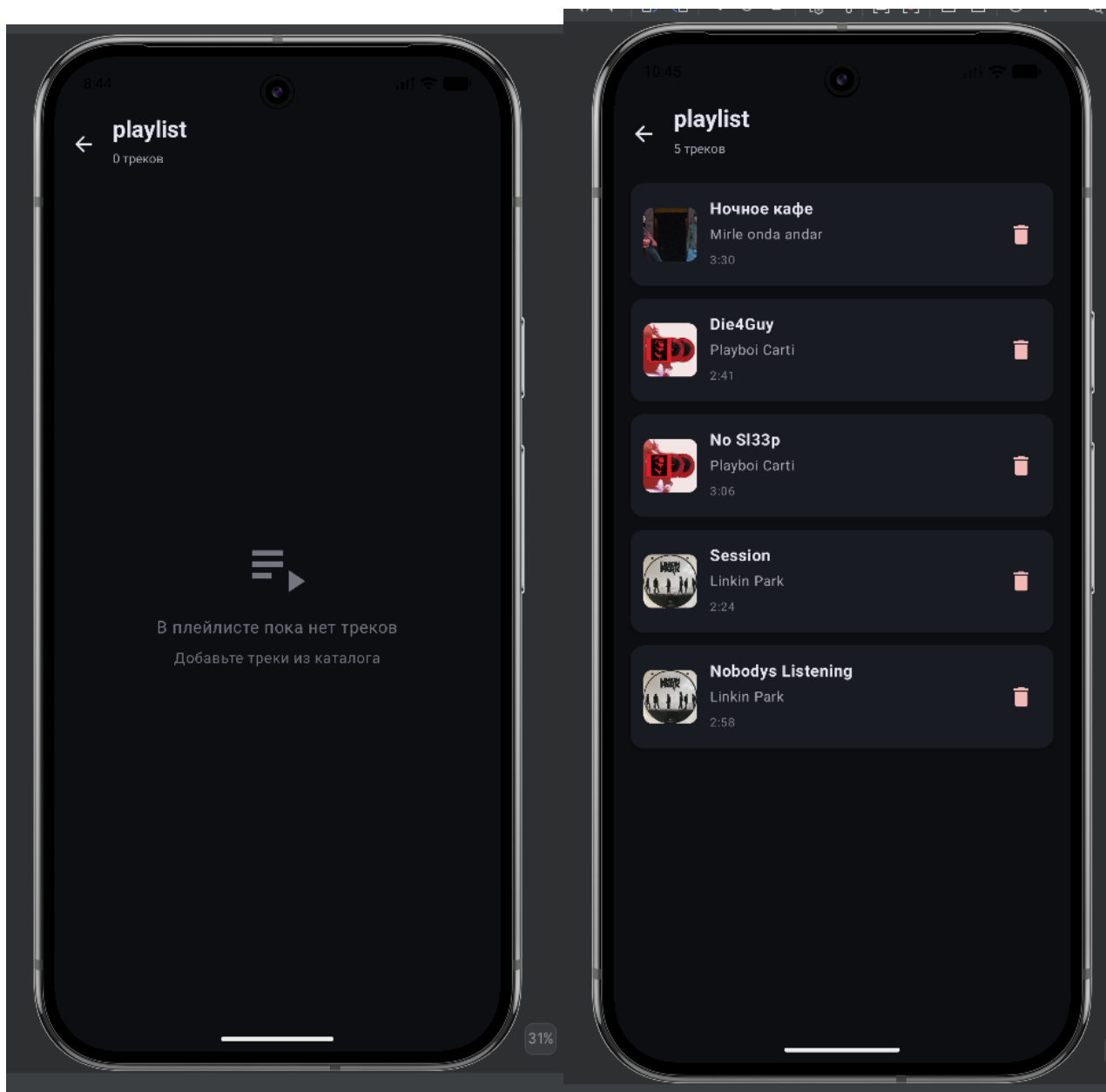


Рисунок 2.8. – Экран детального просмотра плейлиста

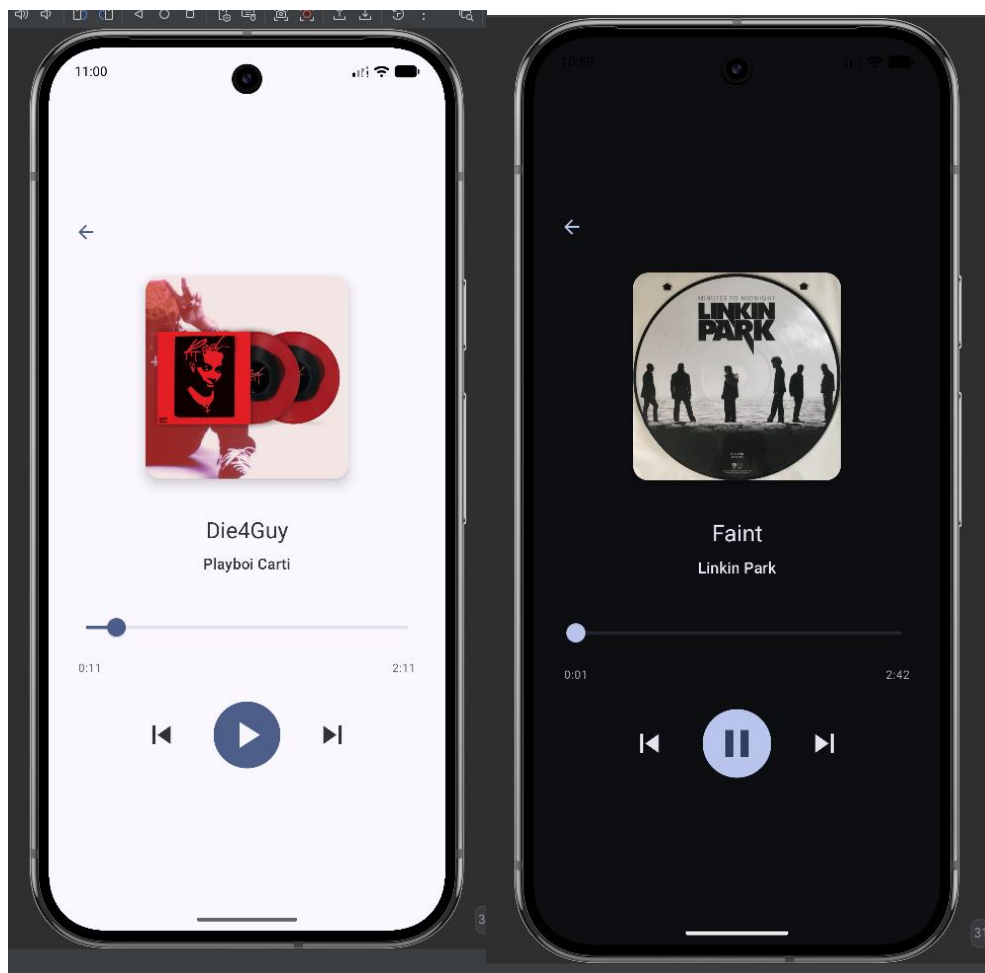


Рисунок 2.9. – Экран плеера

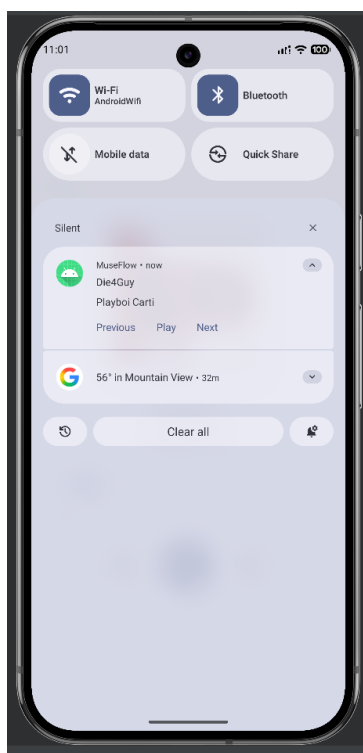


Рисунок 2.10. – Медиа уведомление

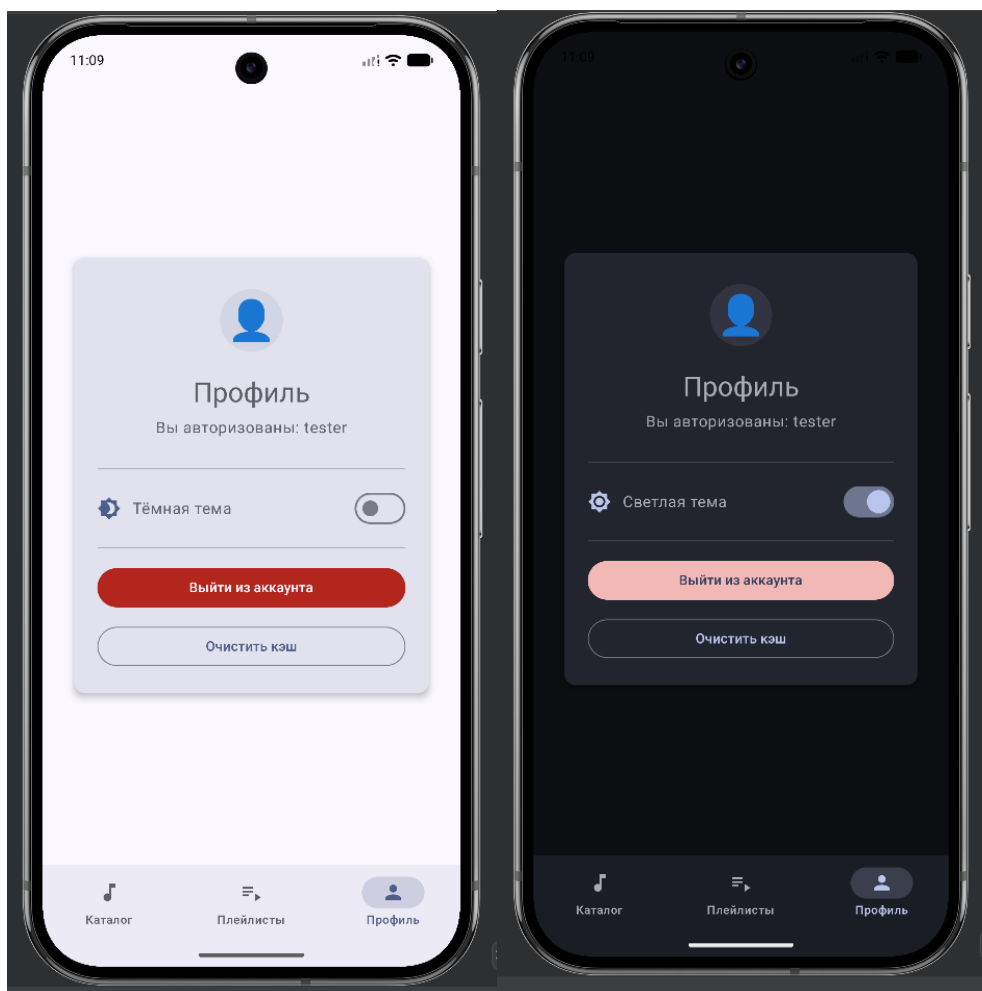


Рисунок 2.11. – Экран профиля (информация, переключатель темы, кнопки)

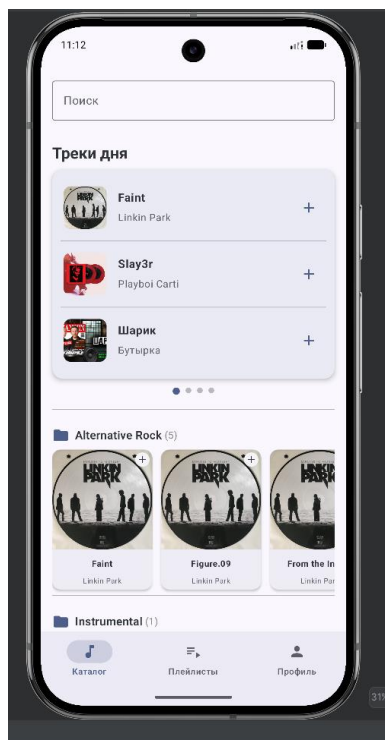


Рисунок 2.12. – Экран каталога без интернета

2.3. Логическая модель приложения

Логическая модель базы данных, разработанная для приложения «MuseFlow», представлена на рисунке 2.20.

Логическая модель базы данных для мобильного приложения «MuseFlow» была спроектирована с целью создания четкой и структурированной основы для управления музыкальным контентом. Она позволяет определить ключевые сущности, такие как пользователи, треки, плейлисты, а также их взаимосвязи. Это помогает обеспечить целостность данных, исключить дублирование информации и повысить надёжность хранимых данных.

Логическая структура, реализованная в базе данных, охватывает ключевые аспекты функциональности приложения:

- регистрация и авторизация пользователей с использованием JWT-токенов;
- получение и кэширование каталога треков с возможностью поиска по названию или исполнителю;
- создание, редактирование и удаление пользовательских плейлистов;
- добавление и удаление треков из плейлистов с сохранением порядка следования;
- хранение метаданных треков (название, исполнитель, жанр, длительность, обложка, аудиофайл).

2.3.1 Проектирование мобильного приложения

Методология IDEF0 применяется для моделирования функциональной структуры системы. Она позволяет визуализировать ключевые процессы, определить их входы, выходы, механизмы и управляющие воздействия. С помощью диаграмм IDEF0 можно наглядно отразить, какие функции выполняет музыкальное приложение, как они взаимодействуют между собой и с внешними объектами.

Контекстная диаграмма описывает приложение как единый процесс, взаимодействующий со своими внешними сущностями (рисунок 2.13). В

качестве входов выступают: «поисковый запрос», «логин и пароль», «название плейлиста», «выбранный трек». В качестве выходов: «JWT-токен», «аудиопоток», «список треков», «список плейлистов». Механизмы работы задают: PostgreSQL + Room БД, Ktor-сервер и мобильное приложение (клиент). Управление обеспечивается: JWT-политикой безопасности, правилами работы с плейлистами и стратегией кэширования.

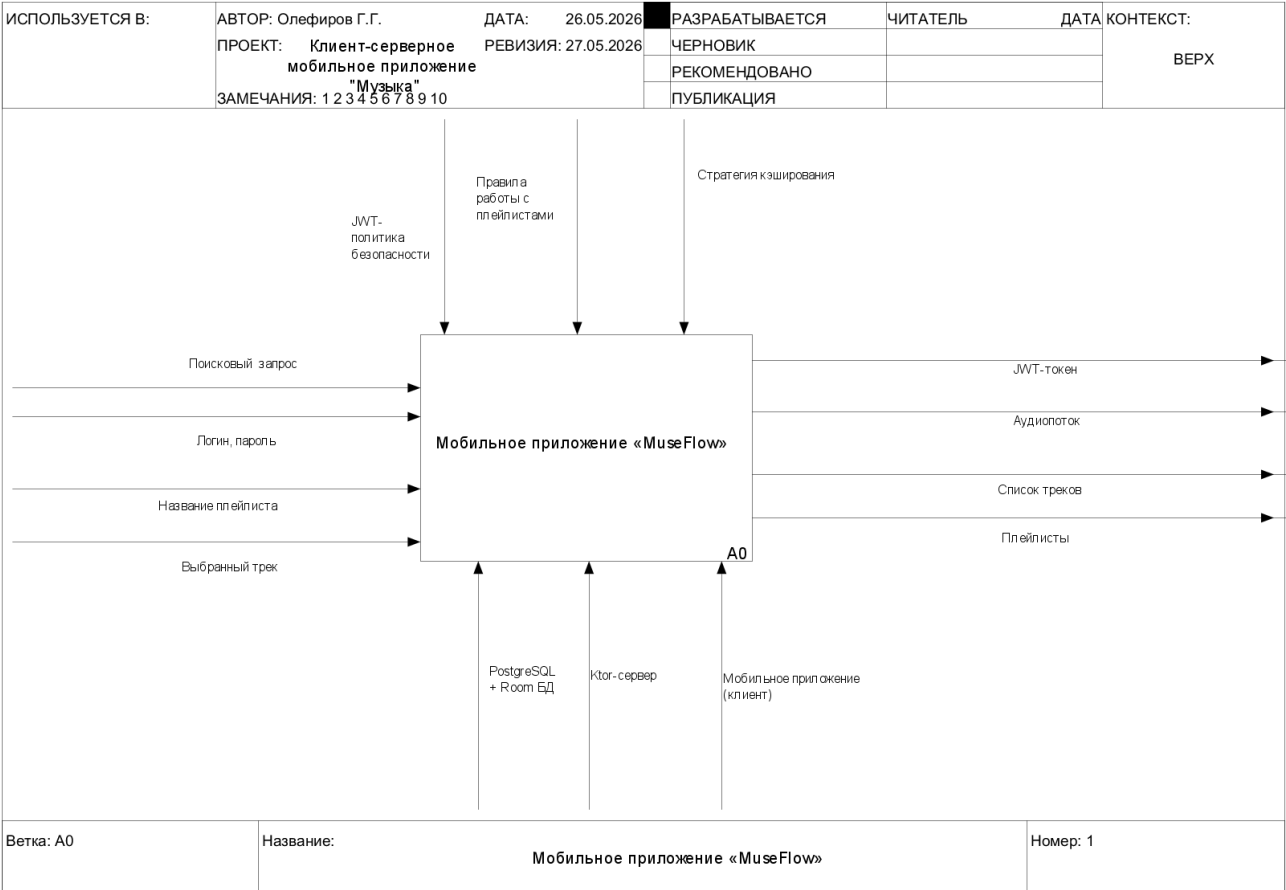


Рисунок 2.13. Контекстный уровень диаграммы IDEF0

Декомпозируем контекст на четыре ключевых подпроцесса: «Регистрация и авторизация» (A1), «Управление каталогом треков» (A2), «Работа с плейлистами» (A3) и «Воспроизведение музыки» (A4)

Процесс A1 получает на вход логин и пароль. Управление осуществляется JWT-политикой безопасности. Механизмами служат PostgreSQL + Room БД, мобильное приложение-клиент и Ktor-сервер. На выходе формируется JWT-токен, который передаётся в A2.

Процесс A2 получает на вход поисковый запрос и авторизованного пользователя (JWT-токен) из A1. Управление осуществляется стратегией кэширования. Механизмами служат PostgreSQL + Room БД, мобильное приложение-клиент и Ktor-сервер. На выходе формируется список треков, который передаётся в A3 и A4.

Процесс A3 получает на вход название плейлиста и список треков из A2. Управление осуществляется стратегией кэширования. Механизмами служат PostgreSQL + Room БД, мобильное приложение-клиент и Ktor-сервер. На выходе формируется список плейлистов, который передаётся в A4.

Процесс A4 получает на вход выбранный трек, список треков из A2 и список плейлистов из A3. Управление осуществляется стратегией кэширования. Механизмами служат мобильное приложение-клиент и медиаплеер ExoPlayer. На выходе формируется аудиопоток.

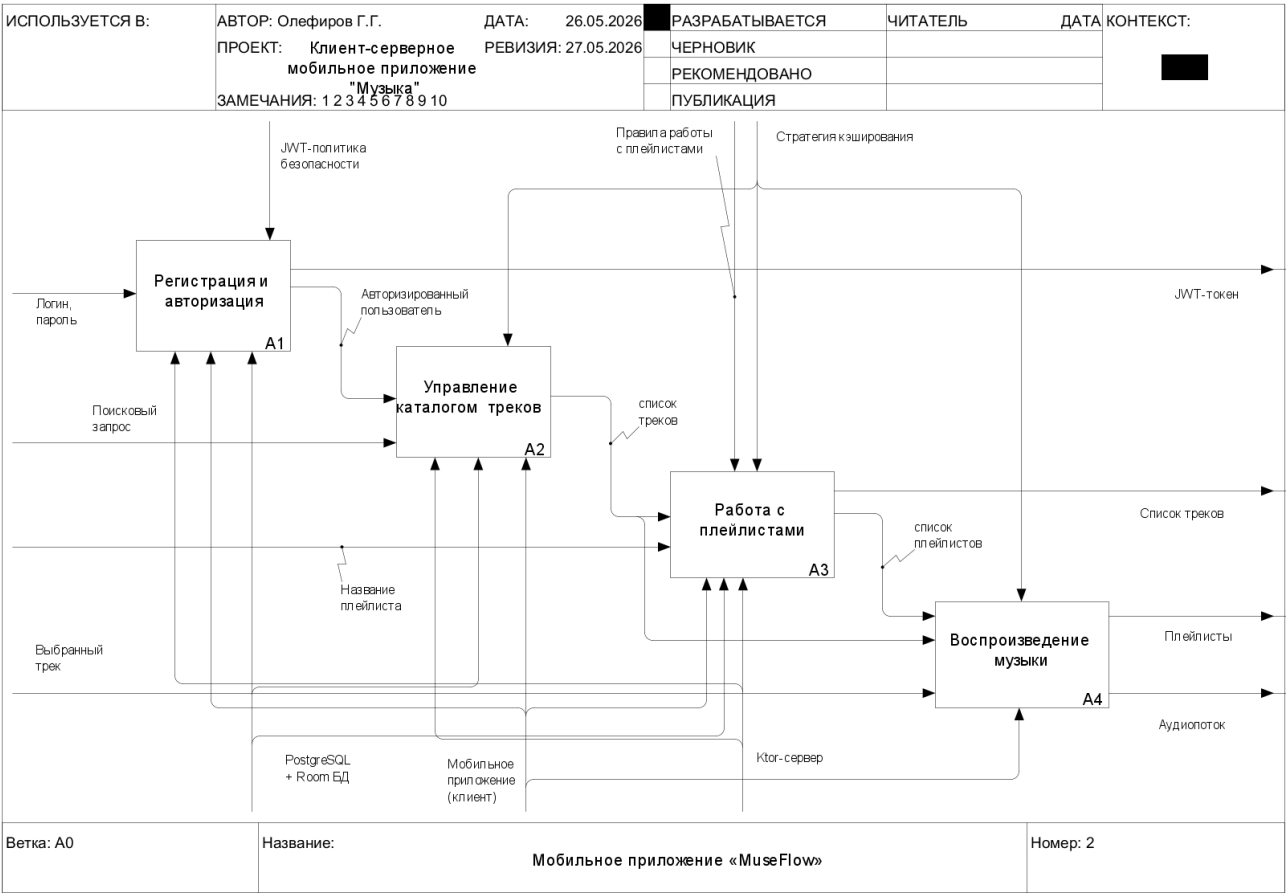


Рисунок 2.14. Декомпозиция контекстного уровня (A0)

На уровне A1 представлена декомпозиция блока «Регистрация и авторизация» (рисунок 2.15). В него входят три подпроцесса: A1.1 «Регистрация», A1.2 «Сохранение пользователя» и A1.3 «Авторизация».

Подпроцесс A1.1 получает на вход логин и пароль, проверяет их на соответствие правилам и передаёт зарегистрированного пользователя в A1.2. Механизмами служат PostgreSQL + Room БД и мобильное приложение-клиент.

Подпроцесс A1.2 сохраняет данные пользователя в базу данных с хешированием пароля (BCrypt). Механизмами служат PostgreSQL + Room БД, Ktor-сервер и мобильное приложение-клиент. Управление осуществляется JWT-политикой безопасности.

Подпроцесс A1.3 выполняет авторизацию: проверяет учётные данные и выдаёт JWT-токен. Механизмами служат PostgreSQL + Room БД, Ktor-сервер и мобильное приложение-клиент. Управление осуществляется JWT-политикой безопасности. На выходе формируется авторизованный пользователь и JWT-токен.

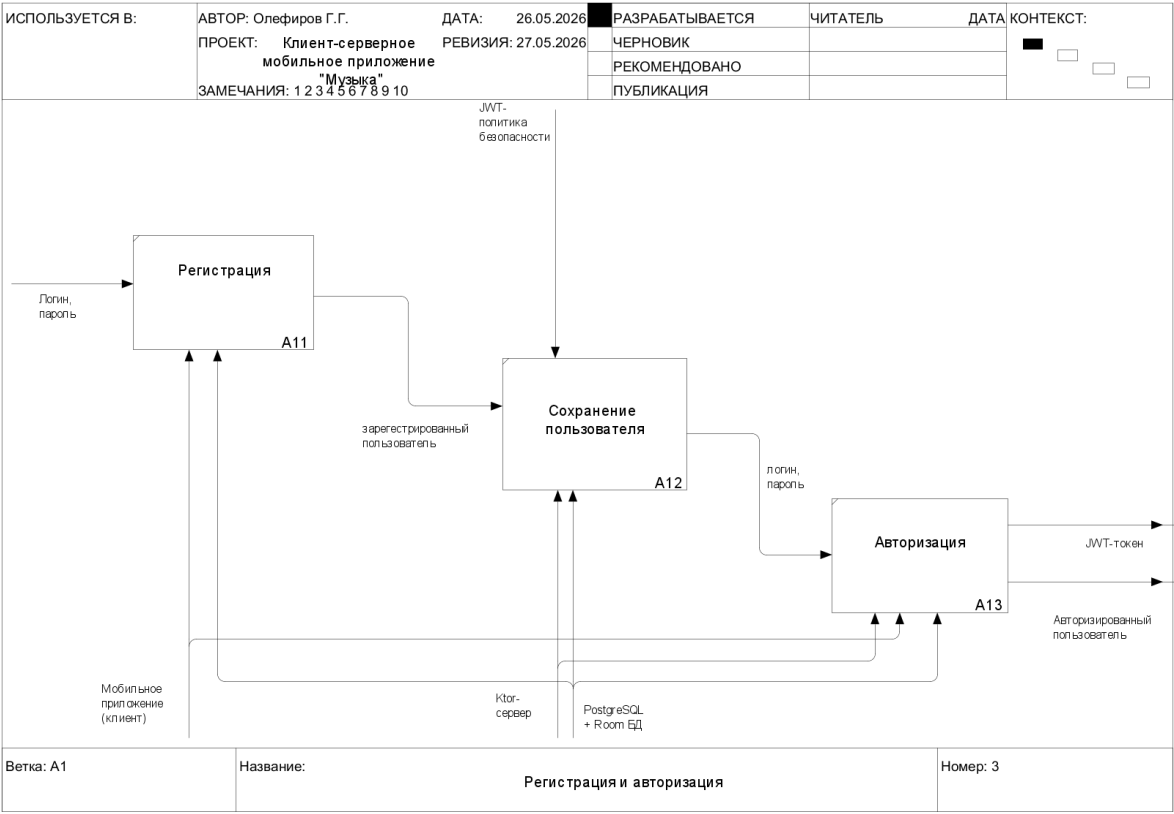


Рисунок 2.15. Декомпозиция процесса A1 «Регистрация и авторизация»

На уровне A2 представлена декомпозиция блока «Управление каталогом треков». Он включает три подпроцесса: A2.1 «Загрузка треков с сервера», A2.2 «Кэширование» и A2.3 «Поиск треков».

Подпроцесс A2.1 получает на вход авторизованного пользователя (JWT-токен) и загружает актуальный список треков с сервера. Механизмами служат Ktor-сервер, PostgreSQL + Room БД и мобильное приложение-клиент.

Подпроцесс A2.2 кэширует полученные треки в локальную базу данных Room для последующего доступа в офлайн-режиме. Механизмами служат PostgreSQL + Room БД и мобильное приложение-клиент. Управление осуществляется стратегией кэширования. На выходе формируется статус кэша, который передаётся в A2.3.

Подпроцесс A2.3 получает на вход поисковый запрос от пользователя и статус кэша из A2.2. При наличии интернета выполняется поиск на сервере, при его отсутствии – по кэшированным данным. Механизмами служат Ktor-сервер и мобильное приложение-клиент. Управление осуществляется стратегией кэширования. На выходе формируется список треков.

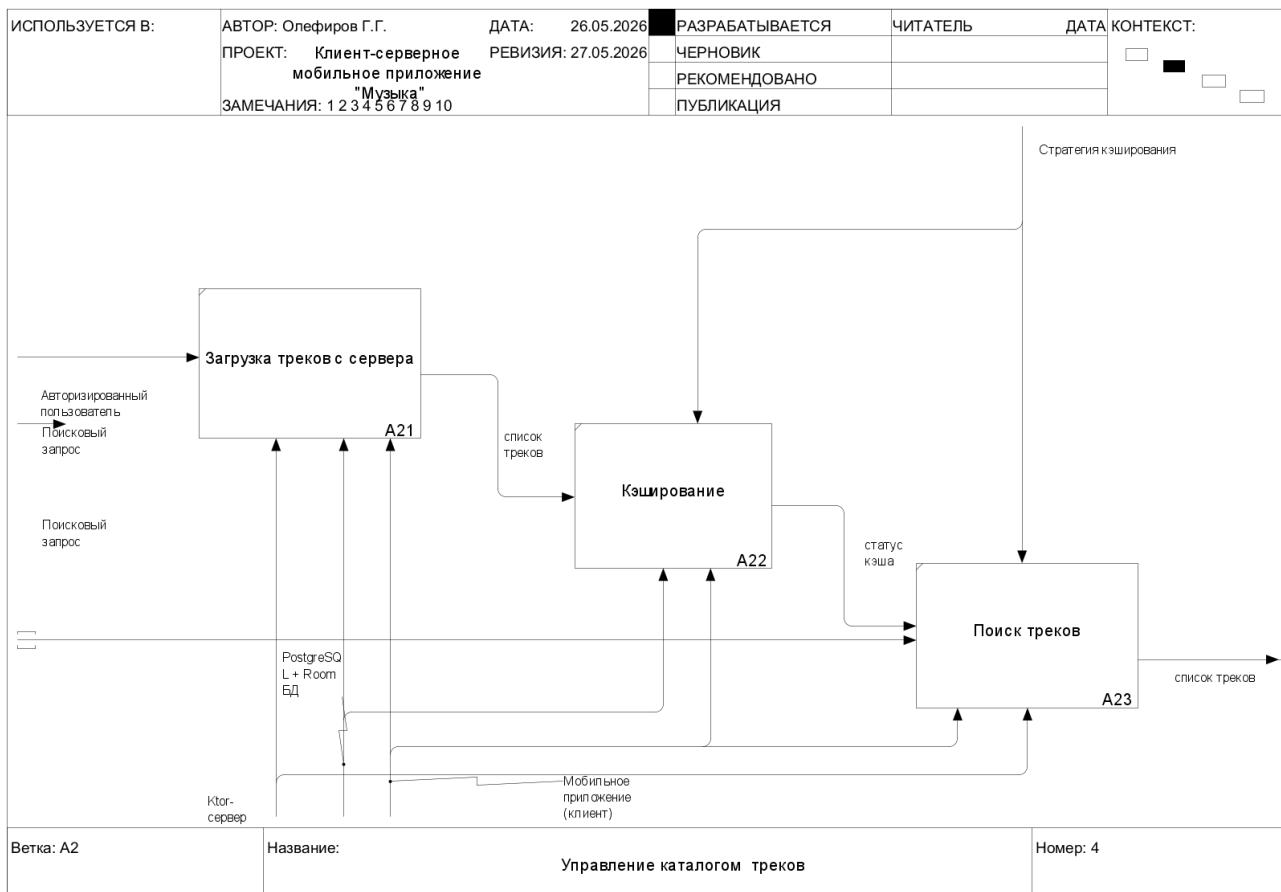


Рисунок 2.16. Декомпозиция процесса A2 «Управление каталогом треков»

На уровне A3 представлена декомпозиция блока «Работа с плейлистами» (рисунок 2.17). Этот процесс разбит на четыре подпроцесса: A3.1 «Получение списка плейлистов», A3.2 «Создание плейлиста», A3.3 «Удаление плейлиста» и A3.4 «Добавление/удаление треков».

Подпроцесс A3.1 получает на вход JWT-токен (userId) и список треков из A2. Механизмами служат PostgreSQL + Room БД, Ktor-сервер и мобильное приложение-клиент. Управление осуществляется правилами работы с плейлистами и стратегией кэширования. На выходе формируется список плейлистов пользователя, который передаётся в A3.2 и A3.3.

Подпроцесс A3.2 получает название плейлиста от пользователя и авторизованного пользователя из A3.1. При успешном создании новый плейлист передаётся обратно в A3.1 для обновления списка. Механизмами служат Ktor-сервер и мобильное приложение-клиент.

Подпроцесс А3.3 получает идентификатор плейлиста для удаления. Возвращает статус удаления в А3.1. Механизмами служат Ktor-сервер и мобильное приложение-клиент.

Подпроцесс А3.4 получает список треков из А2, выбранный плейлист из А3.1 и новый плейлист из А3.2. После добавления или удаления треков возвращает обновлённый состав плейлиста пользователю. Механизмами служат Ktor-сервер и мобильное приложение-клиент. Управление осуществляется правилами работы с плейлистами.

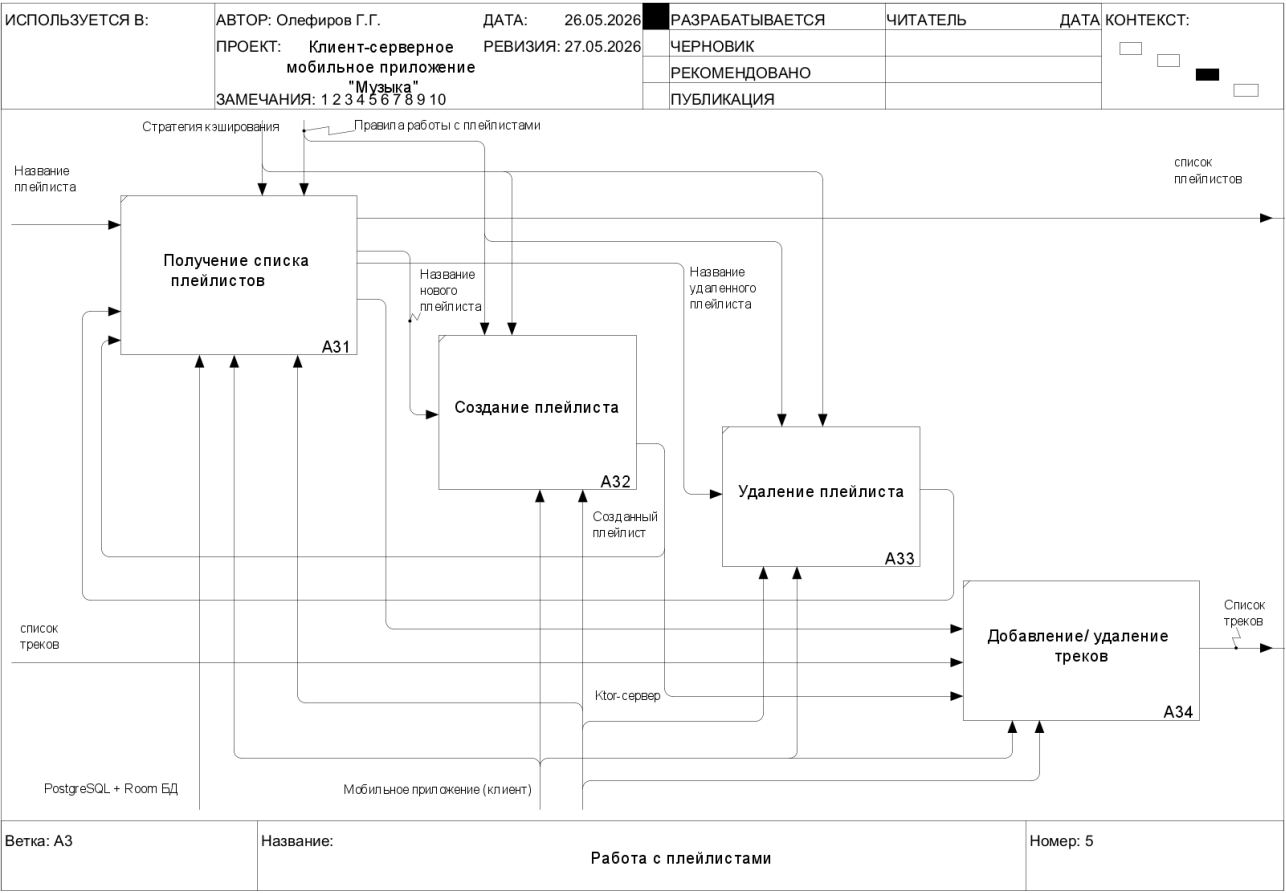


Рисунок 2.17. Декомпозиция процесса А3 «Работа с плейлистами»

На уровне А4 представлена декомпозиция блока «Воспроизведение музыки» (рисунок 2.18). Он состоит из трёх подпроцессов: А4.1 «Загрузка аудио», А4.2 «Управление воспроизведением» и А4.3 «Навигация по плейлисту».

Подпроцесс А4.1 получает на вход выбранный пользователем трек. Механизмом выполнения служит мобильное приложение-клиент с медиаплеером ExoPlayer. На выходе формируется MediaItem, который передаётся в А4.2.

Подпроцесс А4.2 получает на вход команды пользователя (play, pause, next, previous), MediaItem из А4.1, а также обновлённую позицию из А4.3. Управление осуществляется стратегией кэширования. Механизмом выполнения служит мобильное приложение. На выходе формируются аудиопоток (направляется пользователю) и запрос на следующий или предыдущий трек (направляется в А4.3).

Подпроцесс А4.3 получает на вход список плейлистов и список треков из А3, а также запрос на смену трека из А4.2. Управление осуществляется стратегией кэширования. На выходе формируются обновлённые плейлисты с новой позицией воспроизведения, которые передаются обратно в А4.2.

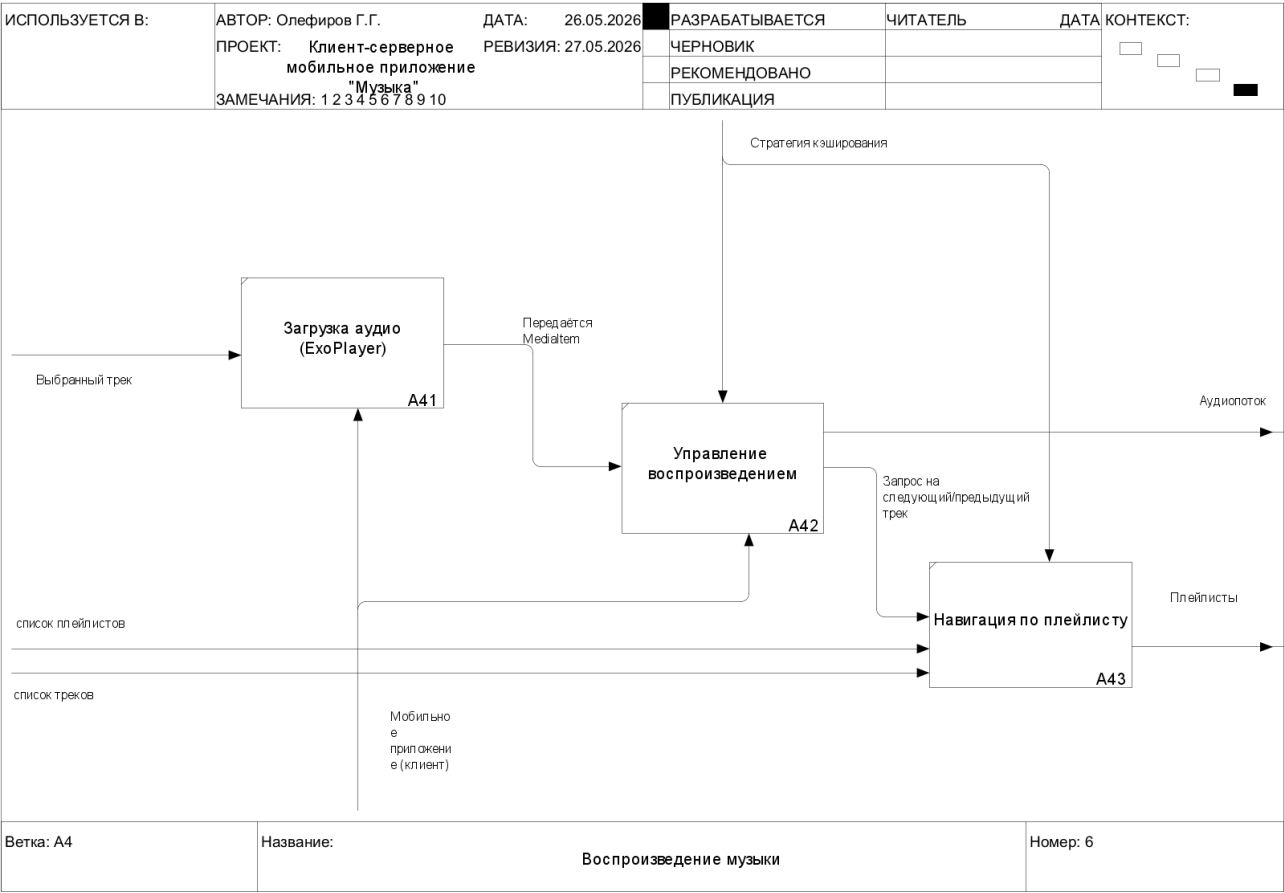


Рисунок 2.18. Декомпозиция процесса А4 «Воспроизведение музыки»

Диаграмма классов

Диаграмма классов является центральной диаграммой логической модели, отображающей статическую структуру системы: классы, их атрибуты, методы и взаимосвязи.

Основные классы системы:

Класс User (Пользователь). Представляет зарегистрированного пользователя. Атрибуты: id (Int, первичный ключ), login (String, уникальный), email (String, уникальный), passwordHash (String), createdAt (LocalDateTime).

Класс Track (Трек). Представляет музыкальную композицию. Атрибуты: id (Int, первичный ключ), title (String), artist (String), duration (Int), coverUrl (String), audioUrl (String), genre (String?). Реализует интерфейс Parcelable для передачи между экранами.

Класс Playlist (Плейлист). Представляет коллекцию треков. Атрибуты: id (Int, первичный ключ), name (String), coverUrl (String?), tracks (List<Track>).

Класс PlaylistTrack (Связь плейлиста и трека). Вспомогательный класс для связи «многие ко многим». Атрибуты: playlistId (Int), trackId (Int), orderIndex (Int).

Класс TokenManager (Менеджер токенов). Управляет аутентификацией. Атрибуты: token (String?), username (String?). Методы: saveToken(), saveUsername(), getToken(), getUsername(), clearToken(). Не хранится в базе данных.

Класс PlaybackService (Сервис воспроизведения). Фоновый сервис для управления музыкой. Атрибуты: player (ExoPlayer), currentTracks (List<Track>), currentTrackIndex (Int). Методы: startPlayback(), play(), pause(), next(), previous(), seekTo(), isPlaying(), getCurrentTrack().

Таблица 3 – Связи между классами

Связь	Тип	Описание
User – Playlist	1 : N	Один пользователь может создать много плейлистов. При удалении пользователя все его плейлисты удаляются каскадно

Playlist – Track	M : N	Многие ко многим (через связующую таблицу PlaylistTrack). Один плейлист может содержать много треков, один трек может входить в много плейлистов
Playlist – PlaylistTrack	1 : N	Один плейлист может иметь много записей в связующей таблице
Track – PlaylistTrack	1 : N	Один трек может иметь много записей в связующей таблице
User – TokenManager	1 : 1	Один пользователь имеет один активный токен (хранится локально)
PlaybackService – Track	1 : N	Сервис воспроизведения управляет очередью треков

Стереотипы и отношения:

- User – основной класс, от которого зависит владение плейлистами
- Track – независимый класс, не привязан к конкретному пользователю.

Реализует Parcelable

- Playlist – агрегатор треков, принадлежит конкретному пользователю
- PlaylistTrack – ассоциативный класс со стереотипом «связка», хранит порядок треков
- TokenManager – вспомогательный класс для аутентификации (хранит JWT и username)
- PlaybackService – сервис для фонового воспроизведения с использованием ExoPlayer и MediaSession

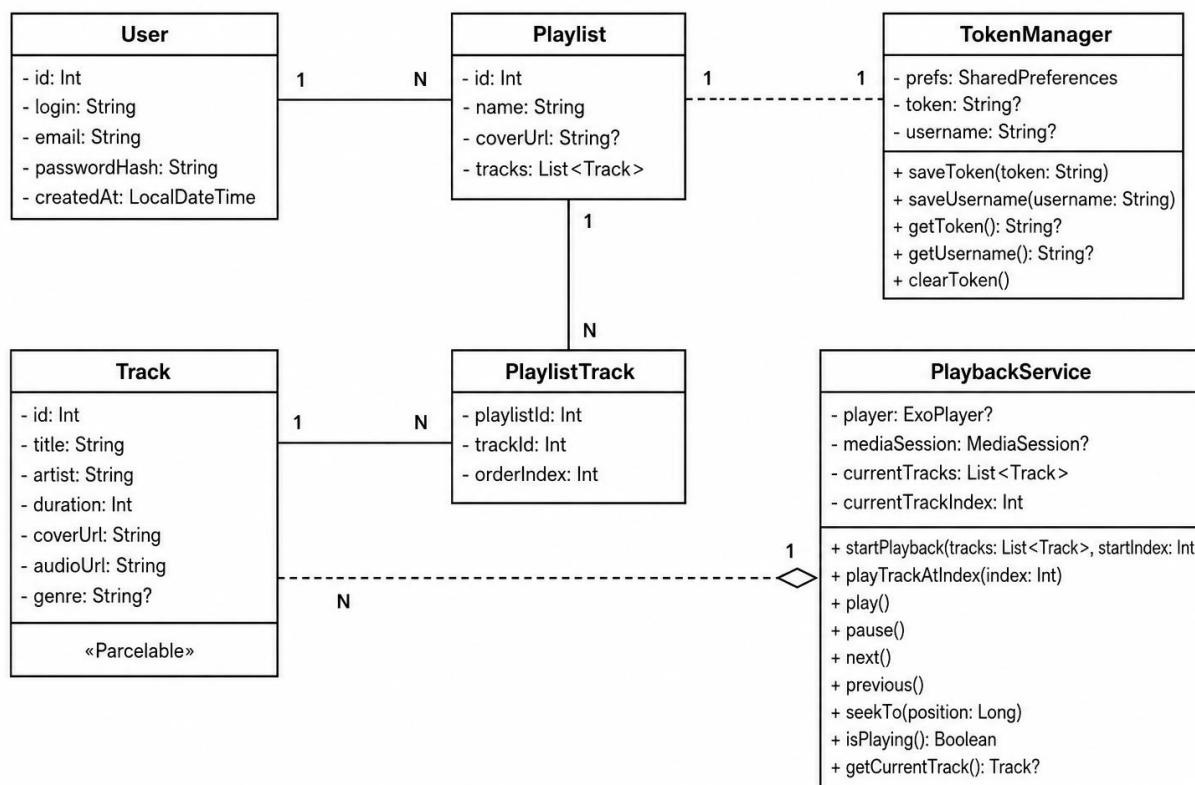


Рисунок 2.19. Диаграмма классов

Таким образом мы смоделировали диаграмму в нотации IDEF0, в которой отображена функциональная структура системы, а также была успешно создана диаграмма классов.

2.3.2 Проектирование базы данных (логическая модель БД, описание каждой таблицы)

Логическая модель базы данных приложения «MuseFlow» включает следующие таблицы:

1. users (пользователи)

Таблица содержит информацию о зарегистрированных пользователях системы.

Таблица 4 –Пользователи

Поле	Тип	Описание
id	SERIAL (PRIMARY KEY)	Уникальный идентификатор пользователя

login	VARCHAR(50) UNIQUE NOT NULL	Уникальное имя пользователя для входа
email	VARCHAR(100) UNIQUE NOT NULL	Адрес электронной почты пользователя
password_hash	VARCHAR(255) NOT NULL	Хешированный пароль (BCrypt)
created_at	TIMESTAMP NOT NULL	Дата и время регистрации

2. tracks (треки)

Таблица хранит метаданные музыкальных композиций.

Таблица 5 – Треки

Поле	Тип	Описание
id	SERIAL (PRIMARY KEY)	Уникальный идентификатор трека
title	VARCHAR(200) NOT NULL	Название трека
artist	VARCHAR(100) NOT NULL	Исполнитель трека
duration	INTEGER NOT NULL	Длительность в секундах
cover_url	VARCHAR(500) NOT NULL	URL-адрес обложки альбома
audio_url	VARCHAR(500) NOT NULL	URL-адрес аудиофайла для воспроизведения
genre	VARCHAR(50)	Жанр музыки (например, Rock, Hip-Hop, Шансон)

3. playlists (плейлисты)

Таблица хранит пользовательские плейлисты.

Таблица 6 – Плейлисты

Поле	Тип	Описание
id	SERIAL (PRIMARY KEY)	Уникальный идентификатор плейлиста
user_id	INTEGER NOT NULL (FOREIGN KEY следует в users.id)	Владелец плейлиста (связь с пользователем)
name	VARCHAR(100) NOT NULL	Название плейлиста

cover_url	VARCHAR(500)	URL-адрес обложки плейлиста (опционально)
created_at	TIMESTAMP NOT NULL	Дата и время создания плейлиста

При удалении пользователя все его плейлисты удаляются автоматически (ON DELETE CASCADE).

4. playlist_tracks (связь плейлистов и треков)

Связующая таблица для реализации отношения «многие ко многим» между плейлистами и треками. Также хранит порядок следования треков в плейлисте.

Таблица 7 – Связь плейлистов и треков

Поле	Тип	Описание
playlist_id	INTEGER NOT NULL (FOREIGN KEY следует в playlists.id)	Идентификатор плейлиста
track_id	INTEGER NOT NULL (FOREIGN KEY следует в tracks.id)	Идентификатор трека
order_index	INTEGER NOT NULL	Порядковый номер трека в плейлисте (для последовательного воспроизведения)

Первичный ключ составной: (playlist_id, track_id).

При удалении плейлиста или трека соответствующие связи удаляются автоматически (ON DELETE CASCADE).

Таблица 8 – Связи между таблицами

Связь	Тип	Описание
users следует в playlists	1 : N	Один пользователь может создать много плейлистов

playlists следует в playlist_tracks	1 : N	Один плейлист может содержать много треков (через связующую таблицу)
tracks следует в playlist_tracks	1 : N	Один трек может входить в много плейлистов (через связующую таблицу)

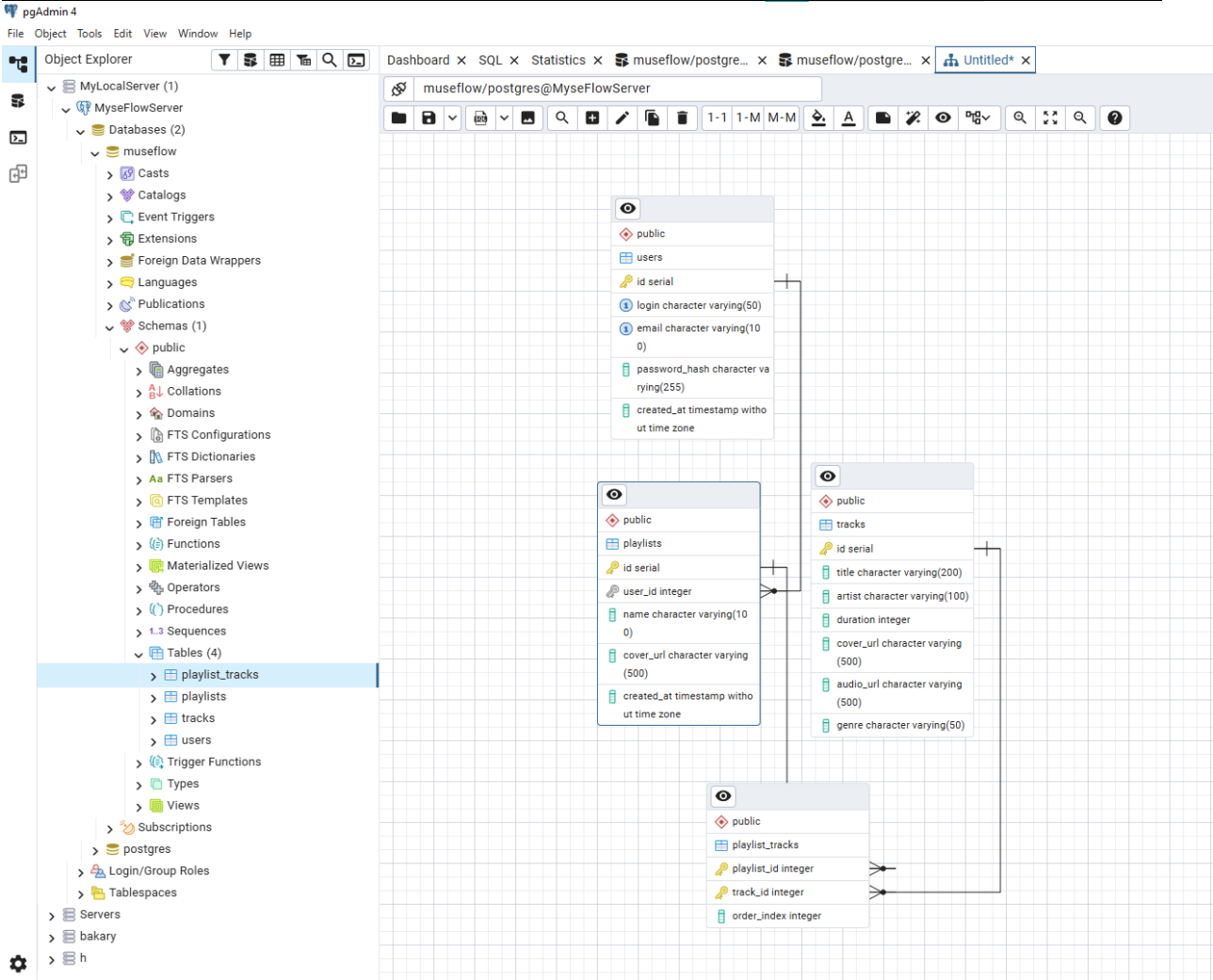


Рисунок 2.20. Логическая модель базы данных

2.3.3 SQL-скрипты для создания таблиц

Листинг 1 – Скрипт

```
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    login VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
```

```

password_hash VARCHAR(255) NOT NULL,
created_at TIMESTAMP NOT NULL
);

CREATE TABLE IF NOT EXISTS tracks (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    artist VARCHAR(100) NOT NULL,
    duration INTEGER NOT NULL,
    cover_url VARCHAR(500) NOT NULL,
    audio_url VARCHAR(500) NOT NULL,
    genre VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS playlists (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
    name VARCHAR(100) NOT NULL,
    cover_url VARCHAR(500),
    created_at TIMESTAMP NOT NULL
);

CREATE TABLE IF NOT EXISTS playlist_tracks (
    playlist_id INTEGER NOT NULL REFERENCES playlists(id) ON DELETE
CASCADE,
    track_id INTEGER NOT NULL REFERENCES tracks(id) ON DELETE
CASCADE,
    order_index INTEGER NOT NULL,

```

```
PRIMARY KEY (playlist_id, track_id)
);

-- Добавляем реальные треки (Linkin Park - альбом Meteora)
INSERT INTO tracks (title, artist, duration, cover_url, audio_url, genre) VALUES
('Faint', 'Linkin Park', 252, 'http://10.0.2.2:8080/uploads/covers/LinkinPark.jpg',
'http://10.0.2.2:8080/uploads/audio/Linkin_Park_-_Faint.mp3', 'Alternative Rock'),
('Figure.09', 'Linkin Park', 197,
'http://10.0.2.2:8080/uploads/covers/LinkinPark.jpg',
'http://10.0.2.2:8080/uploads/audio/Linkin_Park_-_Figure09.mp3', 'Alternative
Rock'),
('From the Inside', 'Linkin Park', 167,
'http://10.0.2.2:8080/uploads/covers/LinkinPark.jpg',
'http://10.0.2.2:8080/uploads/audio/Linkin_Park_-_From_the_Inside.mp3',
'Alternative Rock'),
('Lying from You', 'Linkin Park', 175,
'http://10.0.2.2:8080/uploads/covers/LinkinPark.jpg',
'http://10.0.2.2:8080/uploads/audio/Linkin_Park_-_Lying_from_You.mp3',
'Alternative Rock'),
('Nobodys Listening', 'Linkin Park', 178,
'http://10.0.2.2:8080/uploads/covers/LinkinPark.jpg',
'http://10.0.2.2:8080/uploads/audio/Linkin_Park_-_Nobodys_Listening.mp3',
'Alternative Rock'),
('Session', 'Linkin Park', 144, 'http://10.0.2.2:8080/uploads/covers/LinkinPark.jpg',
'http://10.0.2.2:8080/uploads/audio/Linkin_Park_-_Session.mp3', 'Instrumental');
```


ГЛАВА 3. РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ

3.1 Разработка клиентской части мобильного приложения

Клиентская часть приложения разработана на языке Kotlin с использованием Jetpack Compose и архитектурного паттерна MVVM. В данном разделе описываются ключевые архитектурные решения и фрагменты кода.

3.1.1 Архитектура клиентского приложения

Приложение построено на принципах Clean Architecture в адаптации для Android:

- Presentation Layer (UI): Compose-экраны (Screens), ViewModels, State-классы.
- Domain Layer (Use Cases): бизнес-логика, инкапсулированная в use case'ах.
- Data Layer (Repository + Local/Remote): Retrofit-сервисы, Room DAO, репозитории, DTO и Entity модели.
- DI Layer: Hilt-модули для внедрения зависимостей.

3.1.2 Настройка темы и Material3

Система тем поддерживает светлую и тёмную тему с использованием Material3 ColorScheme.

Листинг 2 – Оформление цветовой схемы для Material Design 3 (Theme.kt)

```
private val DarkColorScheme = darkColorScheme(  
    primary = Purple80,  
    secondary = PurpleGrey80,  
    tertiary = Pink80,  
    background = Color(0xFF121212),  
    surface = Color(0xFF1E1E1E),  
    surfaceVariant = Color(0xFF2D2D2D),  
    onBackground = Color(0xFFE0E0E0),
```

```

        onSurface = Color(0xFFE0E0E0),
        onSurfaceVariant = Color(0xFF9E9E9E)
    )

    private val LightColorScheme = lightColorScheme(
        primary = Purple40,
        secondary = PurpleGrey40,
        tertiary = Pink40,
        background = Color(0xFFFF5F5F5),
        surface = Color(0xFFFFFFFF),
        surfaceVariant = Color(0xFFEEEEEE),
        onBackground = Color.Black,
        onSurface = Color.Black,
        onSurfaceVariant = Color(0xFF5F5F5F)
    )

    @Composable
    fun MuseFlowTheme(
        darkTheme: Boolean = isSystemInDarkTheme(),
        content: @Composable () -> Unit
    ) {
        val colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme

        MaterialTheme(
            colorScheme = colorScheme,
            typography = Typography,
            content = content
        )
    }

```

```
}
```

3.1.3 Слой локального хранения (Room)

Для кэширования треков и обеспечения офлайн-доступа используется Room Database.

Листинг 3 – Room Entity для хранения треков (TrackEntity.kt)

```
@Entity(tableName = "tracks")
data class TrackEntity(
    @PrimaryKey
    val id: Int,
    val title: String,
    val artist: String,
    val duration: Int,
    val coverUrl: String,
    val audioUrl: String,
    val genre: String?
)

fun TrackEntity.toDomain(): Track = Track(
    id = id,
    title = title,
    artist = artist,
    duration = duration,
    coverUrl = coverUrl,
    audioUrl = audioUrl,
    genre = genre
)
```

Листинг 4 – DAO для работы с треками (TrackDao.kt)

```
@Dao
```

```

interface TrackDao {

    @Query("SELECT * FROM tracks")
    suspend fun getAllTracks(): List<TrackEntity>

    @Query("SELECT * FROM tracks WHERE title LIKE '%' || :query || '%' OR  

artist LIKE '%' || :query || '%")
    suspend fun searchTracks(query: String): List<TrackEntity>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(tracks: List<TrackEntity>)

    @Query("DELETE FROM tracks")
    suspend fun deleteAll()

}

```

3.1.4 Слой API (Retrofit)

Для взаимодействия с сервером используется Retrofit с сериализацией через Gson.

Листинг 5 – Retrofit API-интерфейс для музыкального приложения (ApiService.kt)

```

interface ApiService {

    @POST("/login")
    suspend fun login(@Body request: LoginRequest): AuthResponse

    @POST("/register")
    suspend fun register(@Body request: RegisterRequest): AuthResponse

    @GET("/tracks")
    suspend fun getTracks(): List<TrackDto>
}

```

```

@GET("/tracks/search")
suspend fun searchTracks(@Query("q") query: String): List<TrackDto>

@GET("/playlists")
suspend fun getPlaylists(): List<PlaylistDto>

@POST("/playlists")
suspend fun createPlaylist(@Body request: CreatePlaylistRequest): PlaylistDto

@DELETE("/playlists/{id}")
suspend fun deletePlaylist(@Path("id") id: Int): String

@POST("/playlists/{id}/tracks")
suspend fun addTrackToPlaylist(
    @Path("id") playlistId: Int,
    @Body request: AddTrackRequest
): Response<Unit>

@DELETE("/playlists/{id}/tracks/{trackId}")
suspend fun removeTrackFromPlaylist(
    @Path("id") playlistId: Int,
    @Path("trackId") trackId: Int
): String
}

```

3.1.5 Управление токеном и аутентификацией

JWT-токен и имя пользователя сохраняются в SharedPreferences через TokenManager.

Листинг 6 – Менеджер токенов (TokenManager.kt)

```

@Singleton
class TokenManager @Inject constructor(
    @ApplicationContext private val context: Context
) {
    private val prefs: SharedPreferences = context.getSharedPreferences("auth",
Context.MODE_PRIVATE)

    fun saveToken(token: String) {
        prefs.edit().putString("jwt_token", token).apply()
    }

    fun saveUsername(username: String) {
        prefs.edit().putString("username", username).apply()
    }

    fun getToken(): String? = prefs.getString("jwt_token", null)

    fun getUsername(): String? = prefs.getString("username", null)

    fun clearToken() {
        prefs.edit().remove("jwt_token").remove("username").apply()
    }
}

```

3.1.6 CatalogViewModel (управление каталогом)

ViewModel управляет состоянием каталога треков и поиском с использованием StateFlow. Реализована стратегия кэширования: при наличии интернета данные загружаются с сервера, при его отсутствии – из Room.

```
@HiltViewModel
class CatalogViewModel @Inject constructor(
    private val getTracksUseCase: GetTracksUseCase,
    private val searchTracksUseCase: SearchTracksUseCase
) : ViewModel() {
    private val _state = MutableStateFlow<CatalogState>(CatalogState.Loading)
    val state: StateFlow<CatalogState> = _state.asStateFlow()

    private var lastSearchQuery = ""
    private var allTracks: List<Track> = emptyList()

    init {
        loadTracks()
    }

    fun loadTracks() {
        viewModelScope.launch {
            _state.value = CatalogState.Loading
            try {
                allTracks = getTracksUseCase()
                _state.value = CatalogState.Success(allTracks)
            } catch (e: Exception) {
                _state.value = CatalogState.Error(e.message ?: "Ошибка загрузки треков")
            }
        }
    }
}
```

```

fun search(query: String) {
    lastSearchQuery = query
    if (query.isBlank()) {
        _state.value = CatalogState.Success(allTracks)
        return
    }
    viewModelScope.launch {
        _state.value = CatalogState.Loading
        try {
            val results = searchTracksUseCase(query)
            _state.value = CatalogState.Success(results)
        } catch (e: Exception) {
            _state.value = CatalogState.Error(e.message ?: "Ошибка поиска")
        }
    }
}

fun retryLastSearch() {
    search(lastSearchQuery)
}

sealed class CatalogState {
    object Loading : CatalogState()
    data class Success(val tracks: List<Track>) : CatalogState()
    data class Error(val message: String) : CatalogState()
}

```


3.1.7 CatalogScreen (экран каталога)

Декларативный экран каталога треков с поиском, историей поиска, блоками по жанрам и секцией «Треки дня».

Листинг 8 – UI-экран каталога треков с использованием Jetpack Compose (CatalogScreen.kt – основные компоненты)

```
@Composable
fun CatalogScreen(
    onTrackClick: (Track, List<Track>) -> Unit,
    onGenreClick: (String) -> Unit,
    playlists: List<Playlist>,
    onAddToPlaylist: (Int, Int) -> Unit,
    viewModel: CatalogViewModel
) {
    val state by viewModel.state.collectAsState()
    var searchQuery by rememberSaveable { mutableStateOf("") }
    // ... остальная логика

    Column(modifier = Modifier.fillMaxSize()) {
        // Поле поиска с кнопкой очистки
        OutlinedTextField(
            value = searchQuery,
            onValueChange = { query ->
                searchQuery = query
                viewModel.search(query)
                if (query.isNotBlank()) {
                    searchHistoryManager.addQuery(query)
                }
            },
            label = { Text("Поиск") },
```

```

trailingIcon = {
    if (searchQuery.isNotEmpty()) {
        IconButton(onClick = {
            searchQuery = ""
            viewModel.search("")
            keyboardController?.hide()
        }) {
            Icon(Icons.Default.Clear, contentDescription = "Очистить")
        }
    }
}

)

// Основной контент
when (state) {
    is CatalogState.Loading -> CircularProgressIndicator()
    is CatalogState.Error -> {
        Column {
            Text((state as CatalogState.Error).message)
            Button(onClick = { viewModel.retryLastSearch() }) {
                Text("Обновить")
            }
        }
    }
    is CatalogState.Success -> {
        val tracks = (state as CatalogState.Success).tracks
        // Отображение треков по жанрам и секции "Треки дня"
    }
}

```

```

    }

    }

}

```

3.1.8 PlaylistsViewModel (управление плейлистами)

ViewModel управляет созданием, удалением плейлистов, а также добавлением и удалением треков.

Листинг 9 – ViewModel для управления плейлистами (PlaylistsViewModel.kt)

```

@HiltViewModel
class PlaylistsViewModel @Inject constructor(
    private val getPlaylistsUseCase: GetPlaylistsUseCase,
    private val createPlaylistUseCase: CreatePlaylistUseCase,
    private val deletePlaylistUseCase: DeletePlaylistUseCase,
    private val addTrackToPlaylistUseCase: AddTrackToPlaylistUseCase,
    private val removeTrackFromPlaylistUseCase:
RemoveTrackFromPlaylistUseCase
) : ViewModel() {

    private val _state = MutableStateFlow<PlaylistsState>(PlaylistsState.Loading)
    val state: StateFlow<PlaylistsState> = _state.asStateFlow()

    fun loadPlaylists() {
        viewModelScope.launch {
            _state.value = PlaylistsState.Loading
            try {
                val playlists = getPlaylistsUseCase()
                _state.value = PlaylistsState.Success(playlists)
            } catch (e: Exception) {
                _state.value = PlaylistsState.Error(e.message ?: "Ошибка загрузки
плейлистов")
            }
        }
    }
}

```

```

    }
}

fun createPlaylist(name: String) {
    viewModelScope.launch {
        try {
            createPlaylistUseCase(name, null)
            loadPlaylists()
        } catch (e: Exception) {
            _state.value = PlaylistsState.Error(e.message ?: "Ошибка создания
плейлиста")
        }
    }
}

fun deletePlaylist(playlistId: Int) {
    viewModelScope.launch {
        try {
            deletePlaylistUseCase(playlistId)
            loadPlaylists()
        } catch (e: Exception) {
            _state.value = PlaylistsState.Error(e.message ?: "Ошибка удаления
плейлиста")
        }
    }
}

suspend fun addTrackToPlaylist(playlistId: Int, trackId: Int): Boolean {
    val result = addTrackToPlaylistUseCase(playlistId, trackId)

```

```

        if (result) loadPlaylists()
        return result
    }

    fun removeTrackFromPlaylist(playlistId: Int, trackId: Int) {
        viewModelScope.launch {
            try {
                removeTrackFromPlaylistUseCase(playlistId, trackId)
                loadPlaylists()
            } catch (e: Exception) {
                _state.value = PlaylistsState.Error(e.message ?: "Ошибка удаления трека")
            }
        }
    }
}

sealed class PlaylistsState {
    object Loading : PlaylistsState()
    data class Success(val playlists: List<Playlist>) : PlaylistsState()
    data class Error(val message: String) : PlaylistsState()
}

```

3.1.9 PlaybackService (сервис воспроизведения)

Фоновый сервис для воспроизведения музыки, реализованный на базе `ExoPlayer` и `MediaSessionService`. Обеспечивает продолжение воспроизведения при свёрнутом приложении и отображение медиа-уведомления.

Листинг 10 – Сервис воспроизведения музыки (PlaybackService.kt – основные методы)

```

class PlaybackService : MediaSessionService() {
    private var player: ExoPlayer? = null

```

```

private var mediaSession: MediaSession? = null
private var currentTracks: List<Track> = emptyList()
private var currentTrackIndex: Int = 0

companion object {
    const val ACTION_PLAY_PAUSE = "play_pause"
    const val ACTION_NEXT = "next"
    const val ACTION_PREVIOUS = "previous"
}

private fun startPlayback(tracks: List<Track>, startIndex: Int = 0) {
    currentTracks = tracks
    currentTrackIndex = startIndex
    playTrackAtIndex(startIndex)
    startForeground(NOTIFICATION_ID, createNotification())
}

private fun playTrackAtIndex(index: Int) {
    val mediaItems = currentTracks.map { trackItem ->
        MediaItem.Builder()
            .setUri(trackItem.audioUrl)
            .setMediaId(trackItem.id.toString())
            .setMediaMetadata(
                MediaMetadata.Builder()
                    .setTitle(trackItem.title)
                    .setArtist(trackItem.artist)
                    .build()
            )
    }
}

```

```

        .build()
    }
    player?.setMediaItems(mediaItems, index, 0)
    player?.prepare()
    player?.play()
}

fun seekTo(position: Long) { player?.seekTo(position) }
fun isPlaying(): Boolean = player?.isPlaying ?: false
}

```

3.1.10 Навигация

Навигация между экранами реализована с помощью Jetpack Navigation Compose. Единый NavHost расположен в MainActivity и управляет переходами.
Листинг 11 – NavHost для управления навигацией (MainActivity.kt – фрагмент)

```

NavHost(
    navController = navController,
    startDestination = if (isAuthenticated) "main" else "auth",
    modifier = Modifier.fillMaxSize()
) {
    composable("auth") {
        AuthScreen(
            authViewModel = authViewModel,
            onSuccess = { token ->
                isAuthenticated = true
                navController.navigate("main") {
                    popUpTo("auth") { inclusive = true }
                }
            }
        )
    }
}

```

```

    )
}

composable("main") {
    MainScreen(
        onNavigateToPlayer = { track, tracks ->
            currentTrack = track
            playlistTracks = tracks
            navController.navigate("player/${track.id}")
        },
        onNavigateToPlaylist = { playlistId ->
            navController.navigate("playlist/$playlistId")
        },
        onNavigateToGenre = { genre ->
            navController.navigate("genre/$genre")
        }
        // ... остальные параметры
    )
}

composable(
    route = "playlist/{playlistId}",
    arguments = listOf(navArgument("playlistId") { type = NavType.IntType })
) { backStackEntry ->
    val playlistId = backStackEntry.arguments?.getInt("playlistId")
    // ... отображение PlaylistDetailScreen
}

```



```

composable("genre/{genreName}") { backStackEntry ->
    val genreName = backStackEntry.arguments?.getString("genreName") ?: ""
    // ... отображение GenreTracksScreen
}

composable("player/{trackId}") { backStackEntry ->
    val trackId = backStackEntry.arguments?.getString("trackId")?.toIntOrNull()
    // ... отображение PlayerScreen
}
}

```

3.2 Разработка серверной части мобильного приложения

Серверная часть приложения разработана на языке Kotlin с использованием фреймворка Ktor. Сервер реализует RESTful API для взаимодействия с клиентским приложением, обеспечивает аутентификацию пользователей, хранение и выдачу музыкальных треков, а также управление пользовательскими плейлистами.

3.2.1 Аутентификация и авторизация

В приложении реализована JWT-ориентированная аутентификация (JSON Web Token). При успешном входе пользователь получает JWT-токен, который передаётся в заголовке `Authorization: Bearer <token>` при каждом запросе к защищённым ресурсам.

Генерация токена: Токен содержит идентификатор пользователя (`userId`) и логин, подписывается алгоритмом HMAC256 и имеет срок действия 7 дней.

Листинг 12 – Конфигурация JWT-аутентификации (JWTConfig.kt)

```

fun Application.configureJWT() {
    val secret = System.getenv("JWT_SECRET") ?: "MuseFlowSecretKey2025"
    val issuer = "MuseFlowServer"
}

```

```

install(Authentication) {
    jwt("auth-jwt") {
        verifier(
            JWT.require(Algorithm.HMAC256(secret))
                .withIssuer(issuer)
                .build()
        )
        validate { credential ->
            val userId = credential.payload.getClaim("userId").asInt()
            val login = credential.payload.getClaim("login").asString()
            if (userId != null && login != null) {
                JWTPrincipal(credential.payload)
            } else null
        }
    }
}

```

Листинг 13 – Контроллер логина (LoginController.kt)

```

class LoginController(private val call: ApplicationCall) {
    suspend fun performLogin() {
        val receive = call.receive<LoginReceiveRemote>()
        val userDTO = Users.fetchUser(receive.login)

        if (userDTO == null) {
            call.respond(HttpStatusCode.BadRequest, "User not found")
        } else {
            if (PasswordHasher.verify(receive.password, userDTO.passwordHash)) {

```

```

        val token = generateJWT(userDTO.id, userDTO.login)
        call.respond(LoginResponseRemote(token = token))
    } else {
        call.respond(HttpStatusCode.BadRequest, "Invalid password")
    }
}
}

private fun generateJWT(userId: Int, login: String): String {
    val secret = System.getenv("JWT_SECRET") ?: "MuseFlowSecretKey2025"
    return JWT.create()
        .withIssuer("MuseFlowServer")
        .withClaim("userId", userId)
        .withClaim("login", login)
        .withExpiresAt(Date(System.currentTimeMillis() + 7 * 24 * 3600 * 1000))
        .sign(Algorithm.HMAC256(secret))
}
}

```

Хеширование паролей: Пароли пользователей не хранятся в открытом виде. Для их хеширования используется библиотека BCrypt, обеспечивающая криптографическую стойкость.

Листинг 14 – Хеширование паролей (PasswordHasher.kt)

```

object PasswordHasher {
    fun hash(password: String): String {
        return BCrypt.hashpw(password, BCrypt.gensalt())
    }

    fun verify(password: String, hash: String): Boolean {

```

```
        return BCrypt.checkpw(password, hash)
    }
}
```

3.2.2 Основная конфигурация сервера

Сервер настраивается в файле `Application.kt`, где подключаются все необходимые модули: CORS, сериализация, JWT, маршрутизация и статические файлы.

Листинг 15 – Основной файл сервера (Application.kt)

```
package org.example

import io.ktor.http.HttpHeaders
import io.ktor.http.HttpMethod
import io.ktor.server.application.*
import io.ktor.server.engine.embeddedServer
import io.ktor.server.http.content.*
import io.ktor.server.netty.Netty
import io.ktor.server.plugins.cors.routing.CORS
import io.ktor.server.response.*
import io.ktor.server.routing.*
import org.example.features.login.configureLoginRouting
import org.example.features.playlists.configurePlaylistsRouting
import org.example.features.register.configureRegisterRouting
import org.example.features.tracks.configureTracksRouting
import org.example.features.user.configureUserRouting
import org.example.routing.configureRouting
import org.example.routing.configureSerialization
import org.example.utils.configureJWT
```

```

import org.example.utils.configureLogging
import org.jetbrains.exposed.sql.Database
import java.io.File

fun main() {
    Database.connect(
        url = "jdbc:postgresql://localhost:5432/museflow",
        driver = "org.postgresql.Driver",
        user = "postgres",
        password = "1234"
    )

    embeddedServer(Netty, port = 8080, host = "0.0.0.0") {
        install(CORS) {
            allowMethod(HttpMethod.Options)
            allowMethod(HttpMethod.Get)
            allowMethod(HttpMethod.Post)
            allowMethod(HttpMethod.Put)
            allowMethod(HttpMethod.Delete)
            allowHeader(HttpHeaders.Authorization)
            allowHeader(HttpHeaders.ContentType)
            anyHost()
        }

        configureSerialization()
        configureJWT()

        routing {

```

```

// Статические файлы
staticFiles("/uploads/audio", File("uploads/audio"))
staticFiles("/uploads/covers", File("uploads/covers"))

// Для отладки
get("/test-faint") {
    val audioFile = File("uploads/audio/Linkin_Park_-_Faint.mp3")
    call.respondText(
        "Faint - Linkin Park\n" +
            "Audio file exists: ${audioFile.exists()}\n" +
            "Audio path: ${audioFile.absolutePath}\n" +
            "File size: ${if (audioFile.exists()) audioFile.length() else 0}
bytes\n" +
            "Uploads dir exists: ${File("uploads").exists()}\n" +
            "Uploads path: ${File("uploads").absolutePath}"
    )
}

get("/test-figure09") {
    val audioFile = File("uploads/audio/Linkin_Park_-_Figure09.mp3")
    call.respondText(
        "Figure.09 - Linkin Park\n" +
            "Audio file exists: ${audioFile.exists()}\n" +
            "Audio path: ${audioFile.absolutePath}\n" +
            "File size: ${if (audioFile.exists()) audioFile.length() else 0}
bytes"
    )
}

```

//сокращено

```

    configureRouting()
    configureLoginRouting()
    configureRegisterRouting()
    configureTracksRouting()
    configurePlaylistsRouting()
    configureUserRouting()
    configureLogging()
  }.start(wait = true)
}

```

3.2.3 API-эндпоинты

Ниже приведена таблица всех реализованных API-эндпоинтов сервера MuseFlow.

Таблица 10 - API-эндпоинты

Метод	Эндпоинт	Аутентификация	Описание
POST	/login	Нет	Авторизация пользователя, выдача JWT-токена
POST	/register	Нет	Регистрация нового пользователя
GET	/tracks	Bearer token	Получение списка всех треков
GET	/tracks/search?q={query}	Bearer token	Поиск треков по названию или исполнителю
GET	/tracks/{id}	Bearer token	Получение информации

			о конкретном треке
GET	/playlists	Bearer token	Получение всех плейлистов текущего пользователя
POST	/playlists	Bearer token	Создание нового плейлиста
PUT	/playlists/{id}	Bearer token	Обновление названия плейлиста
DELETE	/playlists/{id}	Bearer token	Удаление плейлиста
POST	/playlists/{id}/tracks	Bearer token	Добавление трека в плейлист
DELETE	/playlists/{id}/tracks/{trackId}	Bearer token	Удаление трека из плейлиста
GET	/user/me	Bearer token	Получение информации о текущем пользователе
GET	/uploads/audio/{filename}	Нет	Доступ к аудиофайлам
GET	/uploads/covers/{filename}	Нет	Доступ к обложкам

Пример реализации маршрутов для треков:

Листинг 16 – Маршруты для работы с треками (TracksRouting.kt)

```
package org.example.features.tracks

import io.ktor.http.*
import io.ktor.server.application.*
```



```
import io.ktor.server.auth.*
import io.ktor.server.response.*
import io.ktor.server.routing.*
import org.example.database.tracks.Tracks

fun Application.configureTracksRouting() {
    routing {
        authenticate("auth-jwt") { // Защищённые эндпоинты (требуют токен)
            get("/tracks") {
                val tracks = Tracks.getAll()
                call.respond(tracks)
            }

            get("/tracks/search") {
                val query = call.request.queryParameters["q"] ?: ""
                val results = if (query.isNotBlank()) {
                    Tracks.search(query)
                } else {
                    emptyList()
                }
                call.respond(results)
            }

            get("/tracks/{id}") {
                val id = call.parameters["id"]?.toIntOrNull()
                if (id == null) {
                    call.respond(HttpStatusCode.BadRequest, "Invalid track ID")
                    return@get
                }
            }
        }
    }
}
```

```
    }  
    val track = Tracks.getById(id)  
    if (track == null) {  
        call.respond(HttpStatusCode.NotFound, "Track not found")  
    } else {  
        call.respond(track)  
    }  
}  
  
}  
  
}
```

3.3. Тестирование разработанного приложения

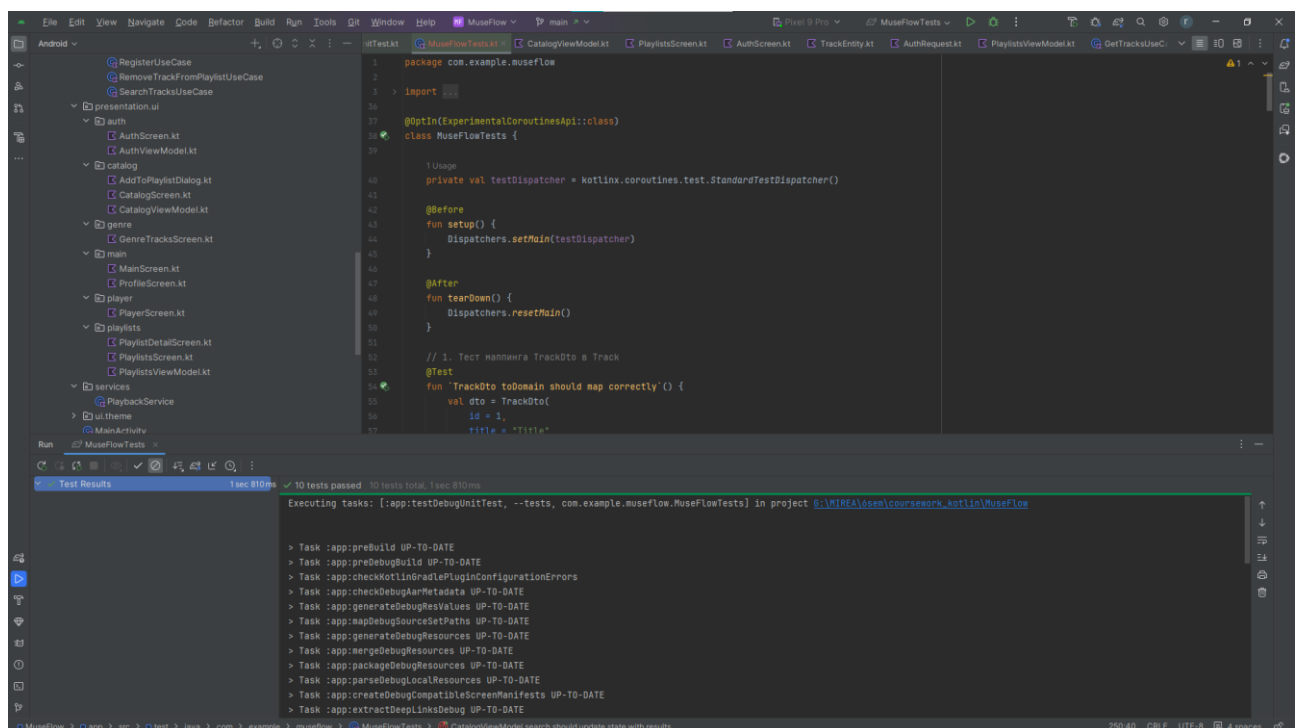


Рисунок 3.1. Успешное выполнение каждого теста

Я подготовил 10 unit-тестов для приложения MuseFlow, охватывающих основные компоненты: репозитории, ViewModel, UseCase и мапперы.

Для написания тестов я добавил в `build.gradle.kts` необходимые библиотеки:

- MockK – для создания моков.
- Kotlinx Coroutines Test – для тестирования корутин и использования `runTest`.
- Turbine – для удобного тестирования `StateFlow`.

Список созданных тестов:

1.TrackDto mapping: Проверка корректности преобразования сетевой модели в доменную.

2.TracksRepositoryImpl (Success): Проверка того, что данные загружаются из API и сохраняются в кэш.

3.TracksRepositoryImpl (Cache Fallback): Проверка переключения на локальную БД при ошибке сети (IOException).

4.TracksRepositoryImpl (Error): Проверка выброса исключения, если нет ни интернета, ни кэша.

5.RemoveTrackFromPlaylistUseCase: Тестирование бизнес-логики вызова репозитория.

6.AuthViewModel (Login Success): Проверка перехода состояний Idle -> Loading -> Success.

7.AuthViewModel (Login Error): Проверка обработки ошибок и установки состояния Error.

8.CatalogViewModel (Init Load): Проверка автоматической загрузки треков при инициализации.

9.CatalogViewModel (Grouping): Тестирование логики группировки треков по жанрам.

10.CatalogViewModel (Search): Проверка обновления состояния при выполнении поиска.

Listing 17 - MuseFlowTests.kt.

```
package com.example.museflow
```

```
import app.cash.turbine.test
import com.example.museflow.data.local.dao.TrackDao
import com.example.museflow.data.local.entities.TrackEntity
import com.example.museflow.data.network.api.ApiService
import com.example.museflow.data.network.models.TrackDto
import com.example.museflow.data.repository.TracksRepositoryImpl
import com.example.museflow.data.repository.toDomain
import com.example.museflow.domain.models.Track
import com.example.museflow.domain.repository.PlaylistsRepository
import com.example.museflow.domain.usecase.LoginUseCase
import com.example.museflow.domain.usecase.RegisterUseCase
import com.example.museflow.domain.usecase.RemoveTrackFromPlaylistUseCase
import com.example.museflow.presentation.ui.auth.AuthState
import com.example.museflow.presentation.ui.auth.AuthViewModel
import com.example.museflow.presentation.ui.catalog.CatalogState
import com.example.museflow.presentation.ui.catalog.CatalogViewModel
import com.example.museflow.domain.usecase.GetTracksUseCase
import com.example.museflow.domain.usecase.SearchTracksUseCase
import io.mockk.coEvery
import io.mockk.coVerify
import io.mockk.mockk
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.ExperimentalCoroutinesApi
import kotlinx.coroutines.test.UnconfinedTestDispatcher
import kotlinx.coroutines.test.resetMain
import kotlinx.coroutines.test.runTest
import kotlinx.coroutines.test.setMain
import org.junit.After
import org.junit.Assert.assertEquals
```

```
import org.junit.Assert.assertTrue
import org.junit.Before
import org.junit.Test
import java.io.IOException

@OptIn(ExperimentalCoroutinesApi::class)
class MuseFlowTests {

    private val testDispatcher = kotlinx.coroutines.test.StandardTestDispatcher()

    @Before
    fun setup() {
        Dispatchers.setMain(testDispatcher)
    }

    @After
    fun tearDown() {
        Dispatchers.resetMain()
    }

    // 1. Тест маппинга TrackDto в Track
    @Test
    fun `TrackDto toDomain should map correctly`() {
        val dto = TrackDto(
            id = 1,
            title = "Title",
            artist = "Artist",
            duration = 180,
            coverUrl = "url",
        )
    }
}
```

```

        audioUrl = "audio",
        genre = "Rock"
    )
    val domain = dto.toDomain()

    assertEquals(dto.id, domain.id)
    assertEquals(dto.title, domain.title)
    assertEquals(dto.artist, domain.artist)
    assertEquals(dto.genre, domain.genre)
}

// 2. Тест TracksRepositoryImpl - успешное получение данных с сервера
@Test
fun `TracksRepositoryImpl getTracks should return tracks from API and cache them`() = runTest {
    val api = mockk<ApiService>()
    val dao = mockk<TrackDao>(relaxed = true)
    val repository = TracksRepositoryImpl(api, dao)
    val tracksDto = listOf(TrackDto(1, "T", "A", 100, "c", "a", "g"))

    coEvery { api.getTracks() } returns tracksDto

    val result = repository.getTracks()

    assertEquals(1, result.size)
    assertEquals("T", result[0].title)
    coVerify { dao.insertAll(any()) }
}

```

```

// 3. Тест TracksRepositoryImpl - ошибка сети, загрузка из кэша
@Test
fun `TracksRepositoryImpl getTracks should return tracks from cache when
network fails`() = runTest {
    val api = mockk<ApiService>()
    val dao = mockk<TrackDao>()
    val repository = TracksRepositoryImpl(api, dao)
    val cachedTracks = listOf(TrackEntity(1, "Cached", "A", 100, "c", "a", "g"))

    coEvery { api.getTracks() } throws IOException()
    coEvery { dao.getAllTracks() } returns cachedTracks

    val result = repository.getTracks()

    assertEquals(1, result.size)
    assertEquals("Cached", result[0].title)
}

// 4. Тест TracksRepositoryImpl - ошибка сети и пустой кэш
@Test(expected = Exception::class)
fun `TracksRepositoryImpl getTracks should throw exception when network
fails and cache is empty`() = runTest {
    val api = mockk<ApiService>()
    val dao = mockk<TrackDao>()
    val repository = TracksRepositoryImpl(api, dao)

    coEvery { api.getTracks() } throws IOException()
    coEvery { dao.getAllTracks() } returns emptyList()
}

```

```

        repository.getTracks()
    }

// 5. Тест UseCase RemoveTrackFromPlaylistUseCase
@Test
fun `RemoveTrackFromPlaylistUseCase should call repository`() = runTest {
    val repository = mockk<PlaylistsRepository>()
    val useCase = RemoveTrackFromPlaylistUseCase(repository)

    coEvery { repository.removeTrackFromPlaylist(1, 2) } returns Unit

    useCase(1, 2)

    coVerify { repository.removeTrackFromPlaylist(1, 2) }
}

// 6. Тест AuthViewModel - успешный логин
@Test
fun `AuthViewModel login should update state to Success`() = runTest {
    val loginUseCase = mockk<LoginUseCase>()
    val registerUseCase = mockk<RegisterUseCase>()
    val viewModel = AuthViewModel(loginUseCase, registerUseCase)

    coEvery { loginUseCase("user", "pass") } returns "fake_token"

    viewModel.state.test {
        assertEquals(AuthState.Idle, awaitItem())
        viewModel.login("user", "pass")
    }
}

```



```

// C StandardTestDispatcher нужно продвинуть планировщик
testScheduler.advanceUntilIdle()

// Проверим финальное состояние, пропустив промежуточные если они
есть.

var lastState: AuthState = awaitItem()
while (lastState is AuthState.Loading) {
    lastState = awaitItem()
}
assertEquals(AuthState.Success("fake_token"), lastState)
}
}

// 7. Тест AuthViewModel - ошибка логина
@Test
fun `AuthViewModel login should update state to Error on failure`() = runTest {
    val loginUseCase = mockk<LoginUseCase>()
    val registerUseCase = mockk<RegisterUseCase>()
    val viewModel = AuthViewModel(loginUseCase, registerUseCase)

    coEvery { loginUseCase(any(), any()) } throws Exception("Invalid
credentials")

    viewModel.state.test {
        assertEquals(AuthState.Idle, awaitItem())
        viewModel.login("user", "wrong")

        testScheduler.advanceUntilIdle()
    }
}

```

```

    var lastState: AuthState = awaitItem()
    while (lastState is AuthState.Loading) {
        lastState = awaitItem()
    }
    assertTrue(lastState is AuthState.Error)
    assertEquals("Invalid credentials", (lastState as AuthState.Error).message)
}
}

```

// 8. Тест CatalogViewModel - успешная загрузка треков

@Test

```
fun `CatalogViewModel should load tracks on init`() = runTest {
```

```
    val getTracksUseCase = mockk<GetTracksUseCase>()
```

```
    val searchTracksUseCase = mockk<SearchTracksUseCase>()
```

```
    val tracks = listOf(Track(1, "T", "A", 100, "c", "a", "g"))
```

```
    coEvery { getTracksUseCase() } returns tracks
```

```
    val viewModel = CatalogViewModel(getTracksUseCase,
searchTracksUseCase)
```

```
    viewModel.state.test {
```

```
        testScheduler.advanceUntilIdle()
```

```
        var lastState = awaitItem()
```

```
        while (lastState is CatalogState.Loading) {
```

```
            lastState = awaitItem()
```

```
        }
```

```
        assertTrue(lastState is CatalogState.Success)
```

```
        assertEquals(tracks, (lastState as CatalogState.Success).tracks)
```

```

    }
}

// 9. Тест CatalogViewModel - фильтрация по жанрам
@Test
fun `CatalogViewModel getTracksByGenre should group tracks correctly`() =
runTest {
    val getTracksUseCase = mockk<GetTracksUseCase>()
    val searchTracksUseCase = mockk<SearchTracksUseCase>()
    val tracks = listOf(
        Track(1, "T1", "A1", 100, "c1", "a1", "Rock"),
        Track(2, "T2", "A2", 100, "c2", "a2", "Pop"),
        Track(3, "T3", "A3", 100, "c3", "a3", "Rock")
    )

    coEvery { getTracksUseCase() } returns tracks
    val viewModel = CatalogViewModel(getTracksUseCase,
searchTracksUseCase)

    testScheduler.advanceUntilIdle()

    val grouped = viewModel.getTracksByGenre()

    assertEquals(2, grouped["Rock"]?.size)
    assertEquals(1, grouped["Pop"]?.size)
}

// 10. Тест CatalogViewModel - поиск
@Test

```

```

fun `CatalogViewModel search should update state with results`() = runTest {
    val getTracksUseCase = mockk<GetTracksUseCase>()
    val searchTracksUseCase = mockk<SearchTracksUseCase>()
    val searchResults = listOf(Track(1, "Found", "A", 100, "c", "a", "g"))

    coEvery { getTracksUseCase() } returns emptyList()
    coEvery { searchTracksUseCase("query") } returns searchResults

    val viewModel = CatalogViewModel(getTracksUseCase,
searchTracksUseCase)

    viewModel.state.test {
        testScheduler.advanceUntilIdle()
        // Пропускаем все состояния от инициализации
        while (expectMostRecentItem() is CatalogState.Loading) { awaitItem() }

        viewModel.search("query")
        testScheduler.advanceUntilIdle()

        var lastState = awaitItem()
        while (lastState is CatalogState.Loading) {
            lastState = awaitItem()
        }
        assertTrue(lastState is CatalogState.Success)
        assertEquals(searchResults, (lastState as CatalogState.Success).tracks)
    }
}

```

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы была достигнута поставленная цель: разработано и протестировано клиент-серверное мобильное приложение для прослушивания музыки и управления плейлистами «MuseFlow». Все задачи, сформулированные во введении, были успешно решены.

Основные результаты работы:

Проведён анализ предметной области музыкальных приложений. Рассмотрены ключевые процессы: регистрация пользователей, поиск и воспроизведение треков, создание и управление плейлистами, кэширование для офлайн-доступа.

Выполнен сравнительный анализ существующих аналогов (Яндекс.Музыка, VK Музыка, Spotify, Apple Music). Выявлены их недостатки: закрытый исходный код, отсутствие открытого API, привязка к экосистемам, платная подписка. Сформулированы конкурентные преимущества разрабатываемого приложения: открытая архитектура, бесплатное использование, поддержка офлайн-режима через кэширование, современный технологический стек.

Обоснован выбор инструментальных средств. Для клиентской части выбраны: Kotlin, Jetpack Compose, Material3, Retrofit, Room, Hilt, Coil, ExoPlayer. Для серверной части: Kotlin, Ktor, Exposed ORM, PostgreSQL, BCrypt, JWT. Выбор обоснован современностью, производительностью и безопасностью выбранных технологий.

Определены функциональные (регистрация, авторизация, поиск треков с историей, создание/удаление плейлистов, добавление/удаление треков, воспроизведение музыки, переключение темы, очистка кэша) и нефункциональные (производительность, безопасность JWT, офлайн-доступ, удобство интерфейса) требования к приложению.

Спроектирован пользовательский интерфейс: разработана навигационная структура (8 экранов), выполнено детальное описание каждого экрана с

использованием принципов Material Design 3 и поддержкой светлой и тёмной темы.

Разработана логическая модель приложения в нотации IDEF0: контекстная диаграмма (A-0) и декомпозиция (A0–A4), описывающая процессы регистрации и авторизации, управления каталогом треков, работы с плейлистами и воспроизведения музыки.

Спроектирована диаграмма классов (6 основных классов: User, Track, Playlist, PlaylistTrack, TokenManager, PlaybackService) и ER-диаграмма базы данных (4 таблицы с полным описанием атрибутов и связей).

Реализована клиентская часть на языке Kotlin с использованием Jetpack Compose. Приложение поддерживает светлую и тёмную тему, обеспечивает все запланированные функции: регистрацию, авторизацию, поиск треков с историей (до 10 элементов, сохранение в SharedPreferences), добавление треков в плейлисты, создание и удаление плейлистов, воспроизведение музыки через ExoPlayer с фоновым режимом и медиа-уведомлением, офлайн-доступ через Room.

Реализована серверная часть на языке Kotlin с использованием фреймворка Ktor. Разработано RESTful API с 13 эндпоинтами, реализована JWT-аутентификация с использованием BCrypt для хеширования паролей, интеграция с PostgreSQL через ORM Exposed. Сервер также раздаёт статические файлы (аудиотреки и обложки) через статическую маршрутизацию.

Проведено тестирование по 10 ключевым сценариям, все сценарии успешно пройдены. Реализована стратегия кэширования «сначала сервер, при ошибке – кэш», обеспечивающая работу приложения в условиях отсутствия интернет-соединения.

Технологический стек разработанного приложения:

Клиент: Kotlin, Jetpack Compose, Material3, Retrofit, Room, Hilt, Coil, ExoPlayer, ViewModel, Navigation Compose.

Сервер: Kotlin, Ktor, Exposed ORM, PostgreSQL, BCrypt, JWT.

Архитектура: Clean Architecture + MVVM (клиент), RESTful API (сервер).

Соответствие требованиям курсовой работы:

Приложение полностью реализовано на языке Kotlin.

Интерфейс выполнен с использованием Jetpack Compose на русском языке.

Реализованы экраны регистрации, авторизации, поиска.

Реализована поисковая строка с кнопкой «Очистить», историей поиска (до 10 элементов), плейсхолдерами при отсутствии результатов или ошибке, ProgressBar при загрузке.

Реализован переключатель тёмной темы.

Разработан собственный сервер на Ktor с собственной базой данных PostgreSQL (Firebase не использовался).

Разработанное приложение может быть использовано музыкальными сервисами для организации каталога треков и управления пользовательскими плейлистами. Благодаря открытой архитектуре и наличию REST API, приложение может быть расширено и адаптировано под нужды любого музыкального сервиса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Telemedicine Market Size, Share & Trends Analysis Report, 2024–2030 / Grand View Research. – URL: <https://www.grandviewresearch.com/industry-analysis/telemedicine-industry> (дата обращения: 10.03.2025).
2. Федеральный закон от 29.07.2017 № 242-ФЗ «О внесении изменений в отдельные законодательные акты Российской Федерации по вопросам применения информационных технологий в сфере охраны здоровья» // Собрание законодательства РФ. – 2017. – № 31. – Ст. 4774.
3. Паспорт федерального проекта «Создание единого цифрового контура в здравоохранении на основе единой государственной информационной системы в сфере здравоохранения (ЕГИСЗ)» // Министерство здравоохранения РФ. – URL: <https://minzdrav.gov.ru/poleznye-resursy/edinyy-tsifrovoy-kontur> (дата обращения: 12.03.2025).
4. Телемедицина в России 2023–2024: аналитический отчёт / J'son & Partners Consulting. – URL: <https://json.tv/telemeditsina-v-rossii-2023-2024> (дата обращения: 10.03.2025).
5. СберЗдоровье: официальный сайт. – URL: <https://sberhealth.ru> (дата обращения: 15.03.2025).
6. Яндекс.Здоровье: официальный сайт. – URL: <https://health.yandex.ru> (дата обращения: 15.03.2025).
7. DocDoc: сервис записи к врачам. – URL: <https://docdoc.ru> (дата обращения: 15.03.2025).
8. Medsi: мобильное приложение. – URL: <https://medsi.ru/apps> (дата обращения: 15.03.2025).
9. Jemerov D., Isakova S. Kotlin in Action. – Shelter Island: Manning Publications, 2020. – 368 p.

10. Android Developers: Kotlin overview / Google. – URL: <https://developer.android.com/kotlin> (дата обращения: 16.03.2025).
11. Jetpack Compose documentation / Google. – URL: <https://developer.android.com/jetpack/compose> (дата обращения: 16.03.2025).
12. Guide to app architecture / Google. – URL: <https://developer.android.com/topic/architecture> (дата обращения: 16.03.2025).
13. Ktor documentation / JetBrains. – URL: <https://ktor.io/docs> (дата обращения: 17.03.2025).
14. Exposed ORM documentation / JetBrains. – URL: <https://jetbrains.github.io/Exposed> (дата обращения: 17.03.2025).
15. Kotlin Coroutines guide / Kotlin Foundation. – URL: <https://kotlinlang.org/docs/coroutines-overview.html> (дата обращения: 17.03.2025).
16. MySQL 8.0 Reference Manual / Oracle Corporation. – URL: <https://dev.mysql.com/doc/refman/8.0/en> (дата обращения: 18.03.2025).
17. Spring in Action. 6th ed. / Walls C. – Shelter Island: Manning Publications, 2022. – 520 p.
18. Material Design 3 documentation / Google. – URL: <https://m3.material.io> (дата обращения: 18.03.2025).
19. Hilt documentation / Google. – URL: <https://dagger.dev/hilt> (дата обращения: 19.03.2025).
20. Retrofit documentation / Square. – URL: <https://square.github.io/retrofit> (дата обращения: 19.03.2025).
21. Цифровая трансформация здравоохранения в России: проблемы и перспективы / Иванов А.В., Петрова О.С. // Вестник Росздравнадзора. – 2023. – № 4. – С. 45–52.
22. Телемедицина: правовые и организационные аспекты / Смирнова Е.А. // Медицинское право. – 2022. – № 3. – С. 22–28.

23. Coil: image loading for Android / Coil Contributors. – URL: <https://coil-kt.github.io/coil> (дата обращения: 19.03.2025).

24. Navigation Compose documentation / Google. – URL: <https://developer.android.com/jetpack/compose/navigation> (дата обращения: 20.03.2025)

ПРИЛОЖЕНИЯ

Приложение А. Ссылка на серверную часть

https://github.com/oggiorgi/MIREA/tree/main/6sem/coursework_kotlin/museflow-backend

Приложение Б. Ссылка на клиентскую часть

https://github.com/oggiorgi/MIREA/tree/main/6sem/coursework_kotlin/MuseFlow

Приложение В. Ссылка на файлы разметки

https://github.com/oggiorgi/MIREA/blob/main/6sem/coursework_kotlin/MuseFlow/app/src/main/AndroidManifest.xml